Semantic Log Analytics for UAV Flight Logs

Report and concept explainer: Advanced Database Management Systems term project

Sumit Kumar (22CS30056)

Abstract

UAV flight logs are treated as a database problem: a high-rate time series, a text layer of events, and an ontology that maps two autopilots' field names onto shared concepts, all in one PostgreSQL instance with TimescaleDB and pgvector. The report explains each concept in plain English first and the technical detail second. It documents the schema, the columnstore physical design, the semantic layer, hybrid and filtered vector search, a guarded natural-language-to-SQL interface, and the experiments that justify each choice. All figures are taken from the saved result files in `bench/results/` and the running database; every design decision is logged with its evidence in `docs/decisions.md`.

Contents

1 The Big Picture **3**
1.1 Research questions . . . 3
1.2 Gap in existing work . . . 4

2 Core Concepts **4**
2.1 Time series, hypertables and chunks . . . 4
2.2 Columnstore and compression . . . 4
2.3 Continuous aggregates . . . 4
2.4 Embeddings and nearest-neighbour search . . . 4
2.5 The filtering pitfall . . . 5
2.6 Reciprocal rank fusion . . . 5
2.7 Ontology as relational tables . . . 5
2.8 How retrieval and NL-to-SQL are measured . . . 5

3 Data and Environment **5**

4 Contribution 1: Schema and the Narrow Telemetry Table **6**
4.1 What the running database reports . . . 7

5 Contribution 2: Columnstore Physical Design **8**
5.1 Design experiment . . . 8
5.2 Storage at full size . . . 9
5.3 A query against the columnstore . . . 9
5.4 Loading . . . 9

6 Contribution 3: TimescaleDB against DuckDB **10**

7 Contribution 4: The Semantic Layer and Events **10**
7.1 Ontology and field mapping . . . 10
7.2 ArduPilot: a wrong mapping is worse than a gap . . . 11
7.3 Events . . . 12
7.4 Fault injection: do the detectors find faults of known size? . . . 12

8 Contribution 5: Hybrid Search, Vector Indexes and Filtering **13**
8.1 Design . . . 13
8.2 Search quality: the hybrid did not win . . . 14
8.3 Vector indexes . . . 14
8.4 Does data difficulty change the winner? . . . 15
8.5 Filtered search . . . 16
8.6 The planner picks wrong, so the search routes itself . . . 17

9 Contribution 6: Guarded Natural-Language SQL **18**
9.1 Hostile queries . . . 19
9.2 Evaluation: does the semantic layer help? . . . 20

10 Contribution 7: Similar Flights by Behaviour **21**

11 The Application **22**
11.1 Exploring flights . . . 23
11.2 Incident reports with citations . . . 24
11.3 Similar moments . . . 25
11.4 Fleet views . . . 26
11.5 Asking questions . . . 27
11.6 Looking inside the database . . . 28

12 Results Summary **30**

13 Theoretical Comparison Table **31**

14 Design Decisions **31**

15 Glossary **32**

16 Frequently Asked Questions **32**

17 Conclusions **33**

1 The Big Picture

A drone writes a *flight log* on every flight: positions, attitudes, battery voltage, motor commands, and text messages such as warnings. One log is easy to look at. A *collection* of logs is hard, because the data is three things at once.

- **A dense time series.** The corpus has 300 real PX4 flights and 70.7 million telemetry samples.
- **Free text.** 3,401 events: warnings, failure flags, mode changes.
- **Vendor dialects.** PX4 and ArduPilot call the same physical quantity different things.

Today's tools show one flight at a time, yet the questions people ask span many flights and are asked in words (*"which flights lost a motor?"*).

Key concept

Thesis. Treat the whole collection as a database problem. One PostgreSQL instance stores the time series compactly, maps vendor fields to shared meanings, searches events by meaning, and lets a language model answer questions in SQL under strict safety rules. Every design choice is justified by a measurement.

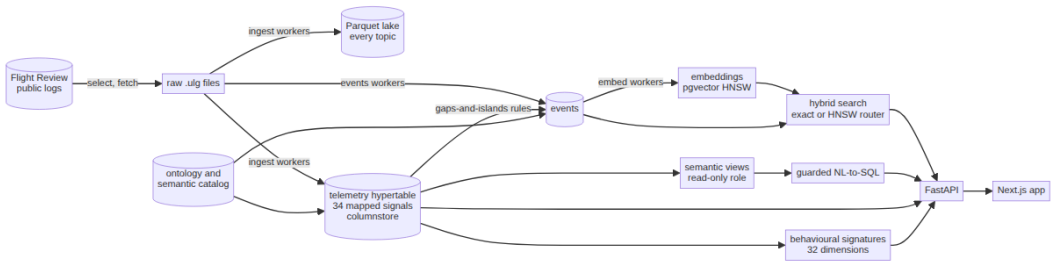


Figure 1: Architecture. Raw logs become a Parquet lake, the canonical telemetry hypertable and events; embeddings and the ontology sit beside them; the application reaches data only through search and guarded read-only views.

1.1 Research questions

1. **RQ1, storage.** How should high-rate telemetry be stored, and what does it cost? (Sections 4, 6)
2. **RQ2, retrieval.** Which vector index and which filtering strategy retrieve events best, and does full-text fusion help? (Section 8)
3. **RQ3, natural-language access.** Can a model write safe, correct SQL, and what does a semantic layer add? (Section 9)
4. **RQ4, detection.** Do the event detectors find faults of known size and duration? (Section 7)

Insight

Negative results are reported as carefully as positive ones. Hybrid text-plus-vector search did *not* beat vector-only search, and the semantic views' gain in NL-to-SQL accuracy was *not* statistically significant. Saying so is part of the method.

1.2 Gap in existing work

Log viewers show one flight and know nothing about meaning. Time-series databases store the numbers but have no ontology or vendor mapping. NL-to-SQL demonstrations exist, but they are rarely guarded or measured. To our knowledge there is no published system that combines an ontology, hybrid retrieval and guarded natural-language SQL over PX4 logs.

2 Core Concepts

2.1 Time series, hypertables and chunks

Plain English: a time series is a list of measurements stamped with a time. A **hypertable** (TimescaleDB) looks like one table but is silently cut into *chunks*, one per time range, so recent data stays small and old data can be stored differently.

Definition

Hypertable: a logical table partitioned by time into chunks. Here, telemetry uses 7-day chunks (134 chunks for this corpus).

2.2 Columnstore and compression

Plain English: a normal table stores row by row. A columnstore stores each column together, so similar numbers sit side by side and compress extremely well, and a query that needs one signal reads only that signal.

Definition

Columnstore with `segmentby`: compressed chunk storage where rows are grouped by the `segmentby` columns and ordered by `orderby`. Here the grouping is (flight, signal), ordered by time, so one compressed row group holds one signal of one flight.

2.3 Continuous aggregates

Plain English: a rollup that the database keeps up to date. Here: min, max and mean per second per flight and signal, used so a zoomed-out chart does not read every sample.

2.4 Embeddings and nearest-neighbour search

Plain English: an embedding turns text into a list of 1,536 numbers so that texts with similar meaning land near each other. Searching by meaning means finding the nearest points. Comparing against every point is exact but slow, so *approximate* indexes trade a little accuracy for speed.

- **HNSW**: a layered graph; hop from node to nearby node toward the query.
- **IVFFlat**: points are sorted into clusters; search only the closest few clusters.
- **DiskANN**: a compressed graph built to live mostly on disk (pgvectorscale).

Math: recall@k is the fraction of the true top-k neighbours the index returns, recall@$k = |A \cap T|/k$, with A the approximate and T the exact result.

2.5 The filtering pitfall

Plain English: pgvector finds the nearest neighbours first and applies the SQL filter afterwards. If the filter keeps one row in a thousand, the index may find 40 neighbours and none of them pass, so the query returns too few rows.

2.6 Reciprocal rank fusion

Plain English: when two rankers disagree (vector similarity versus keyword match), fuse their *ranks*, not their raw scores, because the scores are not comparable. *Math:*

$$\mathrm{RRF}(d) = \sum_i \frac{w_i}{k + \mathrm{rank}_i(d)}$$

with weights w_i and a smoothing constant k (here 60).

2.7 Ontology as relational tables

Plain English: an ontology is a dictionary of concepts and how they relate. Here it is ordinary tables (concept, relation, vendor field, field mapping) walked with recursive queries.

2.8 How retrieval and NL-to-SQL are measured

nDCG@10 ranking quality over the top 10 results, rewarding relevant results near the top.

Execution accuracy (EX) share of generated queries whose *result* equals the result of a reference query. *Strict* requires the same columns and values; *lenient* ignores extra columns.

Relative contrast (RC10) how much farther the 10th neighbour is than the nearest; lower means harder data for any index.

Paired sign test counts questions one condition gets right and another gets wrong, and asks whether the split is plausible under chance.

3 Data and Environment

- **Corpus.** 300 flights chosen (seeded, stratified) from 478,918 public Flight Review PX4 logs (CC-BY); 8.75 GB of ULog files, parsed to a Parquet raw layer and the canonical telemetry table: 70,731,891 rows and 3,401 events.
- **Second autopilot.** An ArduPilot MAVLink log goes through the same ontology.
- **Environment.** 4 cores, 15 GB RAM, no GPU; PostgreSQL 17.11, TimescaleDB 2.30.2, pgvector 0.8.7, pgvectorscale 0.9.1, DuckDB 1.5.6, everything in Docker.
- **Method.** One connection, warm cache, 3 measured passes (vectors) or the median of 5–7 runs (storage); recall against exact search with 200 held-out queries, top 10; a development set and a separate held-out set for NL-to-SQL; bootstrap intervals and paired tests.

```
uavlog=# SHOW ... -- configuration the benchmarks ran under
               name                |   value
-----------------------------------+------------
 effective_cache_size              | 15237128kB
 maintenance_work_mem              | 2031616kB
 max_parallel_maintenance_workers | 2
 max_parallel_workers              | 4
 random_page_cost                  | 1.1
 shared_buffers                    | 5079048kB
 work_mem                          | 31746kB
(7 rows)
```

```
                                                                 version                                                                 ...
-----------------------------------------------------------------------------------------------------------------------------------------...
 PostgreSQL 17.11 (Ubuntu 17.11-1.pgdg22.04+2) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 11.4.0-1ubuntu...
       extname       | extversion
---------------------+------------
 timescaledb         | 2.30.2
 timescaledb_toolkit | 1.26.0
 vector              | 0.8.7
 vectorscale         | 0.9.1
(4 rows)
```

4 Contribution 1: Schema and the Narrow Telemetry Table

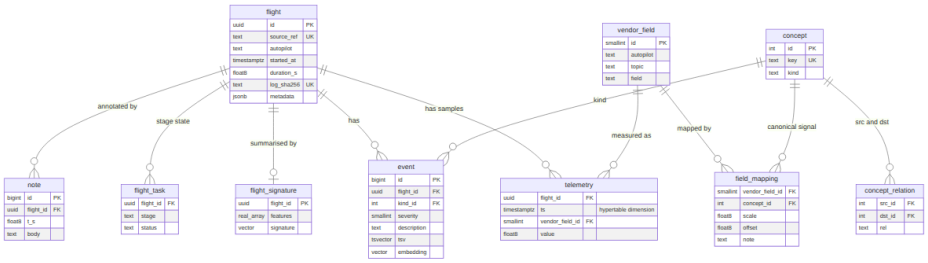


Figure 2: Entity-relationship overview.

Plain English: instead of one column per signal, `telemetry` has one row per (time, flight, field). It is like a ledger: a new kind of expense adds a row, never a column. A new signal, or a whole new autopilot, needs no schema change.

```
    # ---- Telemetry: only ontology-mapped ("hot") fields; full-rate raw stays in Parquet ----
    op.execute("""
    CREATE TABLE telemetry (
        flight_id       uuid     NOT NULL REFERENCES flight(id) ON DELETE CASCADE,
        ts              timestamptz NOT NULL,
        vendor_field_id smallint NOT NULL REFERENCES vendor_field(id),
        value           double precision NOT NULL
    )""")
    op.execute("""
    SELECT create_hypertable('telemetry', by_range('ts', INTERVAL '7 days'))
    """)
    op.execute("CREATE INDEX telemetry_flight_field_ts ON telemetry (flight_id, vendor_field_id, ts DESC)")

    # ---- Semantic view: telemetry expressed in canonical concepts ----
    op.execute("""
    CREATE VIEW canonical_telemetry AS
```

Key concept

Hot fields in the database, the rest in Parquet. Full-rate raw data (60–150 topics per log) stays in Parquet; the database holds the 34 ontology-mapped signals as `(flight_id, ts, vendor_field_id, value)`. The trade-off, recorded in the decision log: a narrow table needs a join to name signals and cannot read two signals in one row without pivoting, but a wide per-topic table fits a changing schema (firmware drift) poorly.

4.1 What the running database reports

```
uavlog=# SELECT hypertable_name, num_dimensions, num_chunks, compression_enabled FROM timescaledb_information...
 hypertable_name | num_dimensions | num_chunks | compression_enabled
-----------------+----------------+------------+---------------------
 telemetry       |              1 |        134 | t
(1 row)

uavlog=# SELECT column_name, time_interval FROM timescaledb_information.dimensions;
 hypertable_name | column_name | time_interval
-----------------+-------------+---------------
 telemetry       | ts          | 7 days
(1 row)

uavlog=# SELECT * FROM timescaledb_information.hypertable_columnstore_settings;
 hypertable |         segmentby         | orderby
------------+---------------------------+---------
 telemetry  | flight_id,vendor_field_id | ts
(1 row)
```

```
uavlog=# SELECT chunk_name, range_start::date, range_end::date, is_compressed FROM timescaledb_information.ch...
     chunk_name     | range_start | range_end  | is_compressed
--------------------+-------------+------------+---------------
 _hyper_1_66_chunk  | 1970-01-01  | 1970-01-08 | t
 _hyper_1_38_chunk  | 1970-01-15  | 1970-01-22 | t
 _hyper_1_90_chunk  | 2005-08-25  | 2005-09-01 | t
 _hyper_1_133_chunk | 2014-12-04  | 2014-12-11 | t
 _hyper_1_158_chunk | 2019-08-22  | 2019-08-29 | t
 _hyper_1_100_chunk | 2021-02-18  | 2021-02-25 | t
 _hyper_1_160_chunk | 2022-04-07  | 2022-04-14 | t
 _hyper_1_162_chunk | 2023-04-13  | 2023-04-20 | t
(8 rows)
```

```
uavlog=# \d+ event
                                                                                    Table "public.event"
     Column      |           Type           | Collation | Nullable |                                      Defaul...
-----------------+--------------------------+-----------+----------+-----------------------------------------...
 id              | bigint                   |           | not null | nextval('event_id_seq'::regclass)       ...
 flight_id       | uuid                     |           | not null |                                         ...
 kind_id         | integer                  |           | not null |                                         ...
 ts_start        | timestamp with time zone |           | not null |                                         ...
 ts_end          | timestamp with time zone |           | not null |                                         ...
 severity        | smallint                 |           | not null | 0                                       ...
 source          | text                     |           | not null |                                         ...
 attrs           | jsonb                    |           | not null | '{}'::jsonb                             ...
 description     | text                     |           | not null |                                         ...
 tsv             | tsvector                 |           |          | generated always as (to_tsvector('englis...
 embedding       | vector(1536)             |           |          |                                         ...
 embedding_model | text                     |           |          |                                         ...
Indexes:
    "event_pkey" PRIMARY KEY, btree (id)
    "event_embedding_hnsw" hnsw (embedding vector_cosine_ops) WITH (m='16', ef_construction='64') WHERE embed...
    "event_flight_ts" btree (flight_id, ts_start)
    "event_kind_ts" btree (kind_id, ts_start)
    "event_tsv" gin (tsv)
Check constraints:
    "event_severity_check" CHECK (severity >= 0 AND severity <= 2)
Foreign-key constraints:
    "event_flight_id_fkey" FOREIGN KEY (flight_id) REFERENCES flight(id) ON DELETE CASCADE
    "event_kind_id_fkey" FOREIGN KEY (kind_id) REFERENCES concept(id)
Access method: heap
```

```
uavlog=# -- indexes of the semantic search tables
         index         | method |    size
-----------------------+--------+------------
 event_embedding_hnsw  | hnsw   | 13 MB
 event_tsv             | gin    | 296 kB
 event_flight_ts       | btree  | 296 kB
 event_pkey            | btree  | 224 kB
 event_kind_ts         | btree  | 224 kB
 flight_signature_hnsw | hnsw   | 136 kB
 flight_metadata_gin   | gin    | 88 kB
 flight_log_sha256_key | btree  | 56 kB
 flight_pkey           | btree  | 40 kB
 flight_signature_pkey | btree  | 32 kB
 flight_started_at     | btree  | 16 kB
 note_pkey             | btree  | 8192 bytes
(12 rows)
```

Found by testing

As a plain rowstore the index (7.6 GB) was larger than the table (4.9 GB). That observation is why the columnstore work matters.

5 Contribution 2: Columnstore Physical Design

```
    # One segment = one signal of one flight, ordered by time: matches the dominant access path
    # (a single flight's single signal over a time range) and compresses well (smooth series).
    op.execute("""
    ALTER TABLE telemetry SET (
        timescaledb.enable_columnstore = true,
        timescaledb.segmentby = 'flight_id, vendor_field_id',
        timescaledb.orderby = 'ts'
    )""")
```

```
    chunks = [r[0] for r in c.execute(
        "SELECT c.relid::regclass::text FROM _timescaledb_catalog.chunk c "
        "JOIN _timescaledb_catalog.hypertable h ON h.id = c.hypertable_id "
        "WHERE h.table_name = 'telemetry' AND (c.status & 1 = 0 OR c.status & 8 = 8) ORDER BY c.id")]
    if a.limit:
        chunks = chunks[: a.limit]
```

Found by testing

Inserting into an already compressed chunk works but leaves it *partial*, with the new rows uncompressed; the first version of the compress stage skipped such chunks. The stage now selects chunks that are uncompressed *or* partial, each in its own transaction, so an interruption loses at most one chunk of work.

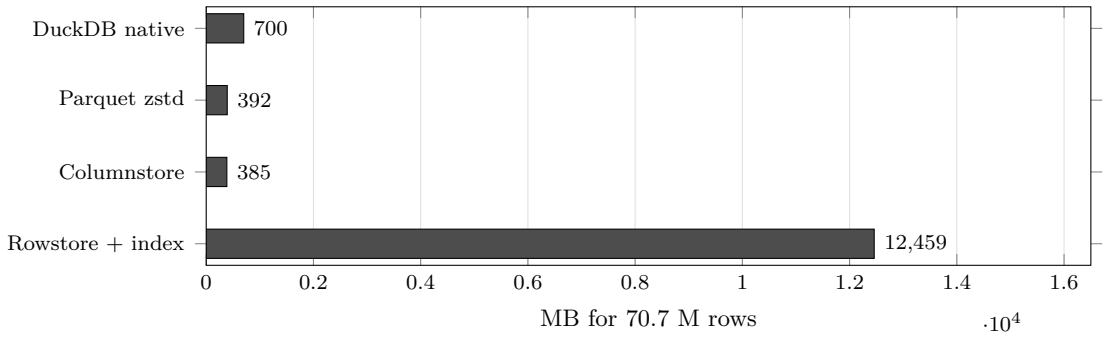
5.1 Design experiment

Variant	Size	Q1	Q2	Q3	Q4
rowstore + B-tree	3,631 MB	10.8 ms	1,519 ms	65 ms	246 ms
segment (flight, signal), order ts	147 MB	12.3	16	10.8	2.7
segment flight only	149 MB	12.9	18.0	11.6	3.2
segment signal only	154 MB	13.0	20.3	14.1	2.2
no segmentby	157 MB	15.0	29.7	13.7	3.6
1-day chunks	150 MB	16.3	15.3	12.3	3.4
30-day chunks	143 MB	9.2	8.3	5.7	2.4

Table 1: 100 flights, 26.9 M rows, median of 7 warm runs. Q1 one signal of one flight; Q2 one signal across all flights; Q3 every signal of one flight; Q4 one signal in a 90-day window.

Reading. Segmenting by flight and signal ordered by time shrinks the data 25 times and makes the cross-flight query (Q2) up to 95 times faster. Design choices change size by under 10%; chunk count and the `segmentby` choice change cross-flight speed about 2 times.

5.2 Storage at full size



The columnstore is 32 times smaller than the rowstore (176 to 5.4 bytes per row), matches sorted zstd Parquet, and is 1.8 times smaller than DuckDB's own file; query results matched the Parquet source.

5.3 A query against the columnstore

```
-- One signal of one flight, as min/max buckets
SELECT time_bucket(make_interval(secs => 1.0), ts) AS bucket, min(value), max(value), avg(value)
FROM canonical_telemetry
WHERE flight_id = '41868bf1-4fcb-4259-a4c5-129603d0f708' AND signal = 'battery.voltage'
GROUP BY 1 ORDER BY 1

Sort  (cost=3259.19..3259.69 rows=200 width=32) (actual time=4.980..5.086 rows=901 loops=1)
  Sort Key: (time_bucket('00:00:01'::interval, t.ts))
  Sort Method: quicksort  Memory: 67kB
  Buffers: shared hit=522
  ->  HashAggregate  (cost=3248.55..3251.55 rows=200 width=32) (actual time=4.575..4.781 rows=901 loops=1)
        Group Key: time_bucket('00:00:01'::interval, t.ts)
        Batches: 1  Memory Usage: 313kB
        Buffers: shared hit=519
        ->  Nested Loop  (cost=10.68..3217.58 rows=1239 width=32) (actual time=0.621..3.351 rows=4502 loops=1)
              Buffers: shared hit=519
              ->  Nested Loop  (cost=0.28..7.36 rows=1 width=18) (actual time=0.034..0.061 rows=2 loops=1)
                    Join Filter: (m.concept_id = c.id)
                    Rows Removed by Join Filter: 50
                    Buffers: shared hit=4
                    ->  Index Scan using field_mapping_pkey on field_mapping m  (cost=0.14..4.22 rows=52 widt...
                          Buffers: shared hit=2
                    ->  Materialize  (cost=0.14..2.37 rows=1 width=4) (actual time=0.000..0.000 rows=1 loops=52)
                          Buffers: shared hit=2
```

Reading. Buffer hits only; a `ColumnarScan` decompresses just the matching segments and the aggregate buckets them into 1-second min/max/mean: 8.4 ms for a 901-bucket envelope.

5.4 Loading

```
def telemetry_table(r: dict, vf_ids: dict, fid: str) -> pa.Table | None:
    t0_us = np.datetime64(r["t0"].replace(tzinfo=None), "us")
    parts = []
    for topic, field, ts_us, vals in r["hot"]:
        ok = np.isfinite(vals)
        parts.append(pa.table({
            "ts": pa.array(t0_us + ts_us[ok].astype("timedelta64[us]"), pa.timestamp("us")),
            "vf": pa.array(np.full(int(ok.sum()), vf_ids[(topic, field)], np.int16)),
            "value": vals[ok],
        }))
    if not parts:
        return None
    t = pa.concat_tables(parts)
```

Columns are canonicalised in Arrow (scale and offset from `field_mapping`) and streamed as CSV into `COPY`: 287 k rows/s into a hypertable, one transaction per flight.

6 Contribution 3: TimescaleDB against DuckDB

Query (p50, ms)	Timescale columnstore	DuckDB over Parquet	DuckDB native
Q1 one signal, one flight, 1 s buckets	25.4	25.3	4.9
Q2 one signal, all flights	69.2	30.6	12.9
Q3 every signal of one flight	18.7	45.7	22.5
Q4 one signal, 90-day window	3.8	28.4	4.5
Q5 count and mean per signal, all data	811.8	1,057.8	478.3
Q6 min/max/mean per flight and signal	1,007.5	2,605.3	2,004.0

Table 2: 70,731,891 rows, median of 5 warm runs, DuckDB with 4 threads (embedded; concurrency not measured).

Load path	Rows per second
Timescale `COPY` (uncompressed hypertable)	286,792
DuckDB row-by-row `INSERT` (200 k rows, for contrast)	603
DuckDB bulk insert from Parquet	6,153,564
DuckDB write to Parquet	4,034,708

Reading. DuckDB wins full scans of a column set (Q5) and point slices on its native file. The columnstore wins pruned access (Q3, Q4) and the whole-corpus summary (Q6), thanks to segment metadata. Postgres adds concurrent writers, transactions, roles and vectors in the *same* engine, so DuckDB stays a comparator.

7 Contribution 4: The Semantic Layer and Events

7.1 Ontology and field mapping

Plain English: PX4 might call battery voltage one thing; ArduPilot another. The ontology gives both a shared name, `battery.voltage`, plus a scale, an offset and a note on the conversion.

Definition

Semantic layer: 63 concepts and 55 relations (34 signals, 7 subsystems, 7 failure modes, 15 event kinds), a field-mapping table (52 mappings), and six documented read-only views.

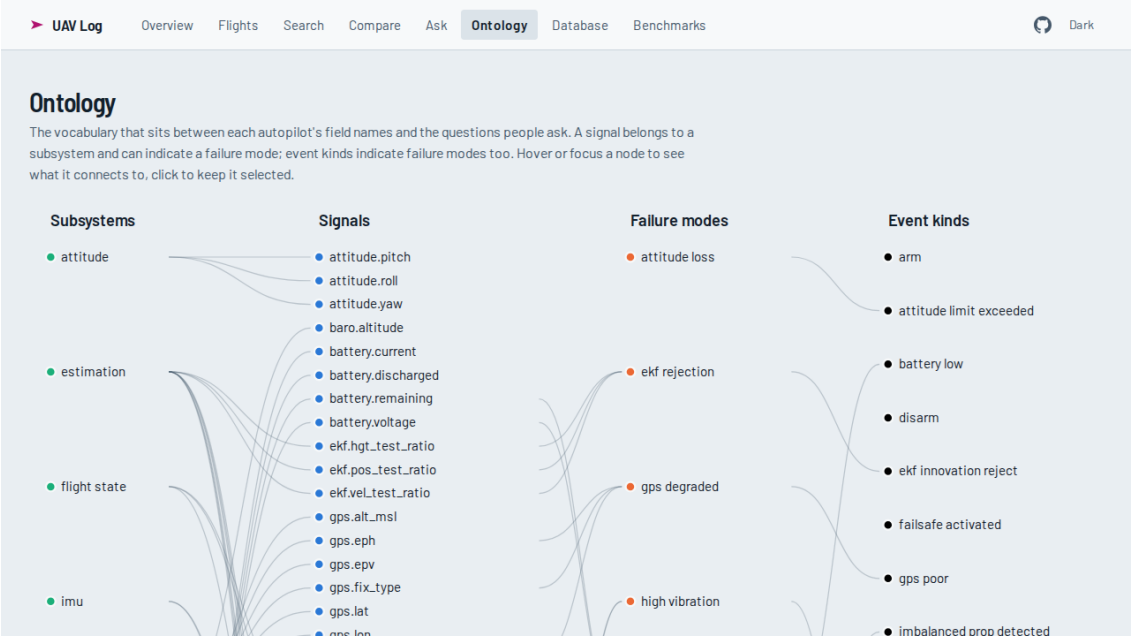


Figure 3: The Ontology page: concepts, relations and mapped vendor fields.

```
uavlog=# -- the documented semantic layer (view comments are the model's schema description)
      view      |                                  comment
----------------+-------------------------------------------------------------------------
 events         | Things that happened during a flight: mode changes, arming, takeoff an
 flights        | One row per flight (a PX4 flight log from Flight Review).
 ontology       | How concepts relate: signal part_of subsystem; signal or event indicat
 signal_summary | Lowest, highest and mean value of every signal in every flight. Use th
 signals        | The canonical telemetry signals (autopilot-independent names).
 telemetry      | Raw time series, about 70 million rows. ALWAYS filter by flight_id and
(6 rows)
```

```
1     CREATE VIEW semantic.events AS
2     SELECT e.id AS event_id, e.flight_id, c.key AS kind,
3            e.severity, CASE e.severity WHEN 2 THEN 'critical' WHEN 1 THEN 'warning' ELSE 'info' END AS
       severity_label,
4            extract(epoch FROM e.ts_start - f.started_at) AS start_s, extract(epoch FROM e.ts_end - f.started_at)
       AS end_s,
5            e.description, e.source, fm.key AS failure_mode
6     FROM event e JOIN concept c ON c.id = e.kind_id JOIN flight f ON f.id = e.flight_id
7     LEFT JOIN concept_relation r ON r.src_id = c.id AND r.rel = 'indicates'
8     LEFT JOIN concept fm ON fm.id = r.dst_id AND fm.kind = 'failure_mode'""")
```

7.2 ArduPilot: a wrong mapping is worse than a gap

```
1 ARDUPILOT_FIELDS = [
2     ("ATTITUDE", "roll", "rad", "attitude.roll", RAD2DEG, 0, "radians to degrees"),
3     ("ATTITUDE", "pitch", "rad", "attitude.pitch", RAD2DEG, 0, "radians to degrees"),
4     ("ATTITUDE", "yaw", "rad", "attitude.yaw", RAD2DEG, 0, "radians to degrees, range -180 to 180"),
5     ("GPS_RAW_INT", "lat", "degE7", "gps.lat", 1e-7, 0, "degrees times 1e7; 0 means no fix and is dropped"),
6     ("GPS_RAW_INT", "lon", "degE7", "gps.lon", 1e-7, 0, "degrees times 1e7; 0 means no fix and is dropped"),
7     ("GPS_RAW_INT", "alt", "mm", "gps.alt_msl", 0.001, 0, "millimetres to metres, above mean sea level"),
8     ("GPS_RAW_INT", "fix_type", "enum", "gps.fix_type", 1, 0, "the same MAVLink enum PX4 uses (3 is a 3D fix)"),
9     ("GPS_RAW_INT", "satellites_visible", "count", "gps.satellites", 1, 0, "satellites VISIBLE; PX4 reports
       satellites USED, so counts run slightly higher; 255 means unknown and is dropped"),
```

Insight

Five ArduPilot groups were deliberately *not* mapped (vibration, motor outputs, EKF ratios, barometric altitude, flight mode) because their meanings differ. A wrong mapping produces confident wrong answers; a gap produces an honest "not available".

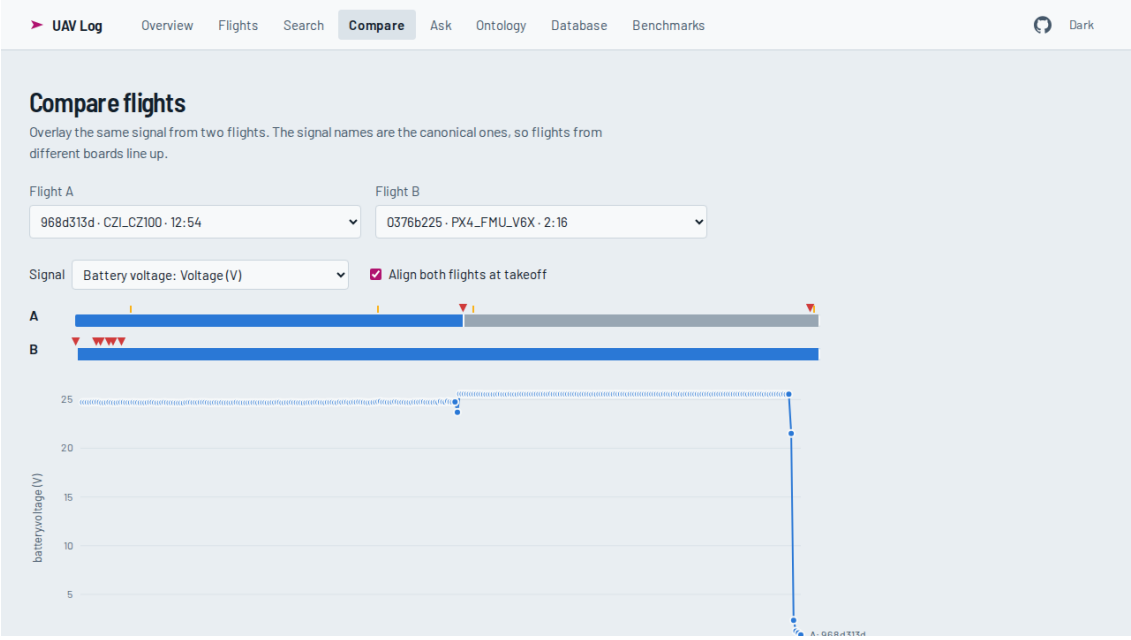


Figure 4: The Compare page overlays one canonical signal from flights on different boards.

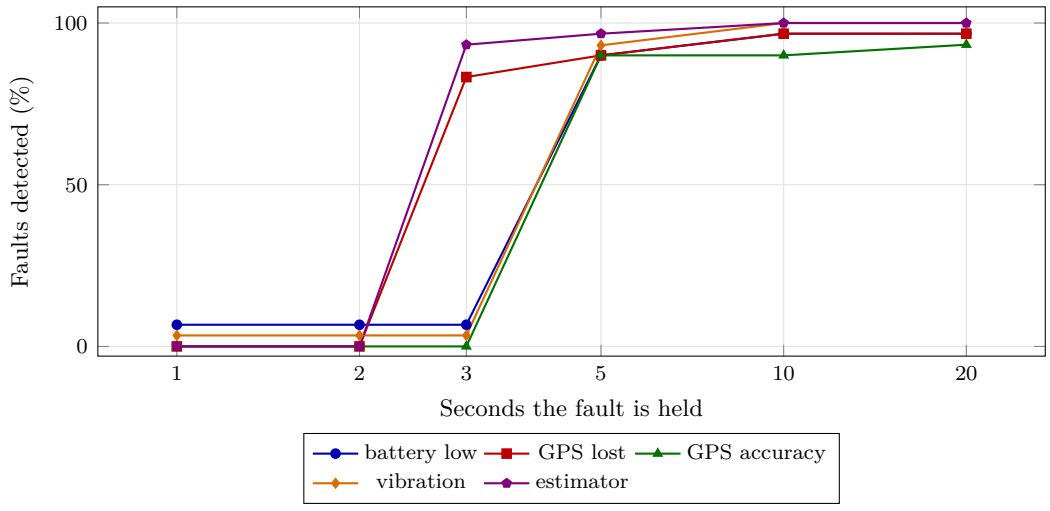
7.3 Events

Events come from log transitions, the failure detector, and *gaps-and-islands* rules over telemetry (window functions that find runs of consecutive abnormal rows). Rules fire only relative to a baseline in the same flight, and near-duplicate messages collapse (3,846 to 3,401).

```
ISLANDS = """
WITH s AS (SELECT ts, value, (value {op} %(thr)s) AS flag FROM {src}
           WHERE flight_id = %(fid)s AND signal = %(sig)s),
g AS (SELECT *, row_number() OVER (ORDER BY ts) - row_number() OVER (PARTITION BY flag ORDER BY ts) AS grp FROM s
      )
SELECT min(ts), max(ts), min(value), max(value) FROM g
WHERE flag AND EXISTS (SELECT 1 FROM s WHERE NOT flag)  -- an event is a deviation from a baseline seen in the
     same flight
GROUP BY grp HAVING max(ts) - min(ts) >= make_interval(secs => %(min_s)s) ORDER BY 1"""
```

7.4 Fault injection: do the detectors find faults of known size?

A constant fault value is held for a stated duration at a random time in the middle of a real flight; the unchanged rule engine then looks for the right event. 30 real flights and 1,043 variants (controls included).



Fault	min	1 s	2 s	3 s	5 s	10 s	20 s	controls
battery critically low	3 s	0.07	0.07	0.07	0.90	0.97	0.97	0.23
GPS lost 3D fix	2 s	0.00	0.00	0.83	0.90	0.97	0.97	0.03
GPS accuracy degraded	3 s	0.00	0.00	0.00	0.90	0.90	0.93	0.03
high vibration	3 s	0.03	0.03	0.03	0.93	1.00	1.00	0.03
estimator rejecting position	2 s	0.00	0.00	0.93	0.97	1.00	1.00	0.07

Table 3: Detection rate by fault duration; "controls" is the false-alarm rate on unmodified flights; "min" is each rule's minimum duration.

Reading. Faults shorter than a rule's minimum duration are missed by design. The few detections at 1–3 s for the battery rule are matches elsewhere in the flight (onset error of 100 to 200 s), not real detections. From 5 s, 90–100% are found, typically within a second of onset. The battery-low rule raises false alarms on 23% of unmodified flights, the main weakness found.

8 Contribution 5: Hybrid Search, Vector Indexes and Filtering

8.1 Design

Event text is embedded (`text-embedding-3-small`, 1,536 dimensions) into a `vector` column with an HNSW index; full text is a generated `tsvector` with GIN. SQL filters cover kind, flight, time and severity. Candidates from each ranker are fused by RRF inside one SQL statement.

```
SQL = """
WITH q AS (SELECT %(vec)s::vector AS v, to_tsquery('english', %(tsq)s) AS tq),
{vec_cte}
fts AS (
    SELECT e.id, row_number() OVER (ORDER BY ts_rank_cd(e.tsv, q.tq) DESC) AS r
    FROM event e, q WHERE e.tsv @@ q.tq {filt}
    ORDER BY ts_rank_cd(e.tsv, q.tq) DESC LIMIT %(pool)s)
SELECT e.id, c.key AS kind, e.severity, e.description, f.source_ref AS flight, f.id::text AS flight_uuid, e.
     ts_start - f.started_at AS t,
       e.embedding <=> (SELECT v FROM q) AS distance,
       %(wv)s * coalesce(1.0 / (%(rrf)s + vec.r), 0) + %(wf)s * coalesce(1.0 / (%(rrf)s + fts.r), 0) AS score,
     vec.r AS vec_rank, fts.r AS fts_rank
FROM vec FULL OUTER JOIN fts ON vec.id = fts.id
JOIN event e ON e.id = coalesce(vec.id, fts.id)
JOIN concept c ON c.id = e.kind_id JOIN flight f ON f.id = e.flight_id
ORDER BY score DESC, vec.r NULLS LAST, fts.r NULLS LAST LIMIT %(k)s"""


def estimated_rows(conn, filt: list[str], params: dict) -> int | None:
```

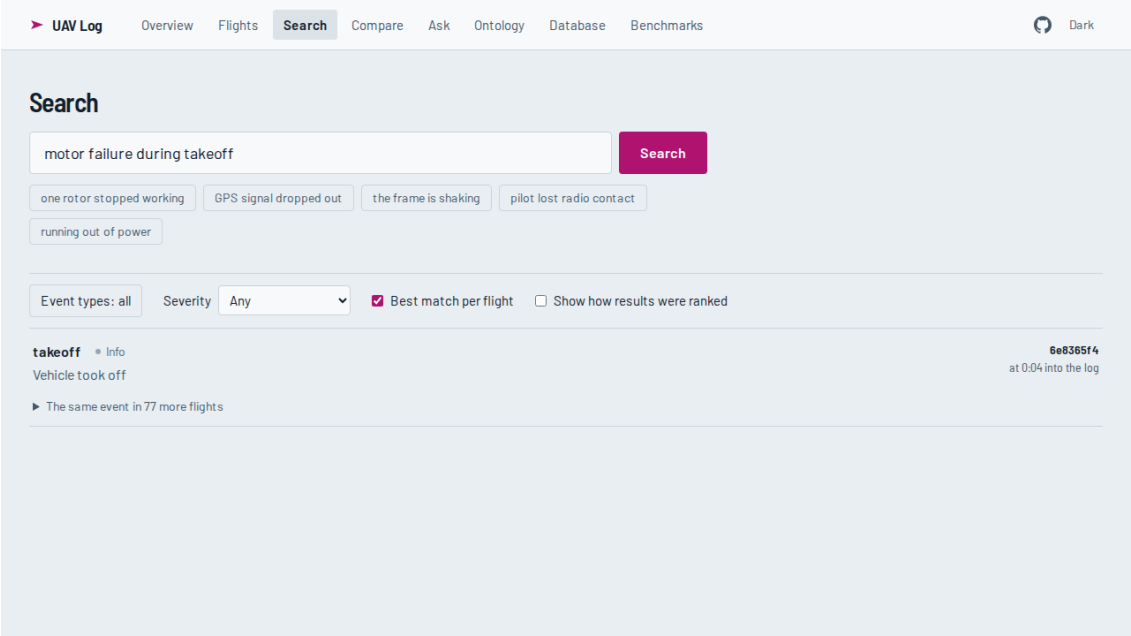


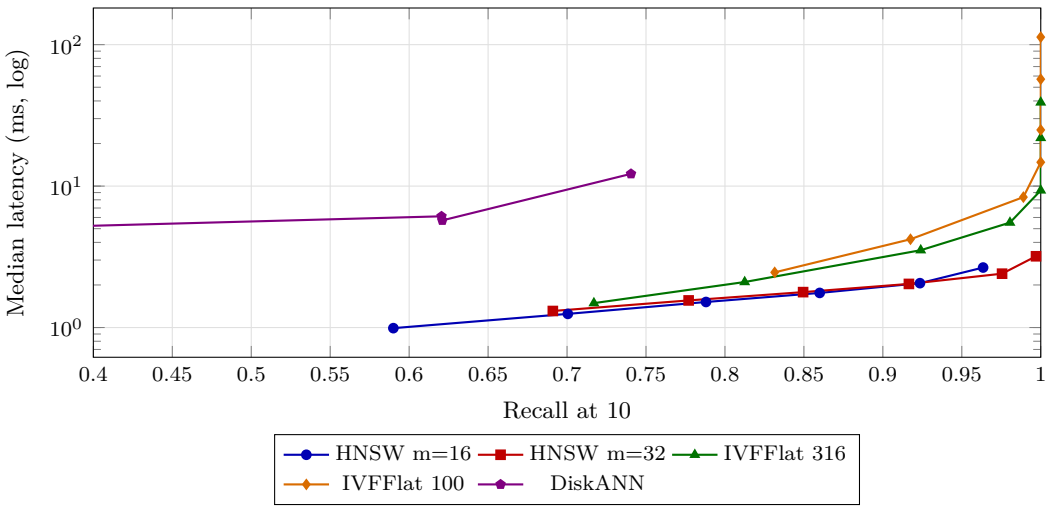
Figure 5: The Search page: hybrid search with ranking details.

8.2 Search quality: the hybrid did not win

Method (65 concept queries, 39 hard)	nDCG@10	hard paraphrases
vector only	0.710	0.59
vector + light full text (weight 0.1)	0.709	0.59
plain RRF	0.600	0.43
full text only	0.48	0.31

Plain fusion is significantly worse than vector only: paired difference -0.109, 95% CI $[-0.172, -0.051]$. Ten exact-term lookups scored 1.000 for every method, so no evidence was found that full text helps even there. Cosine distances sit in a narrow 0.40–0.60 band, so no honest absolute relevance cutoff exists.

8.3 Vector indexes



Index (100,000 vectors)	Build	Size	p50 at recall ≥ 0.95	≥ 0.99
HNSW m=16	56 s	819 MB	2.7 ms	not reached
HNSW m=32	149 s	819 MB	2.4 ms	3.2 ms
IVFFlat, 316 lists	17 s	822 MB	5.5 ms	9.3 ms
IVFFlat, 100 lists	8.5 s	820 MB	8.4 ms	14.8 ms
DiskANN	268 s	68 MB	not reached	not reached (max 0.74)

Table 4: 1,536 dimensions, 200 queries, top 10; exact search takes 362 ms.

HNSW m=32 reaches 0.95 recall in 2.4 ms, over 100 times faster than exact search. IVFFlat builds 3 to 17 times faster; DiskANN is 12 times smaller but loses recall on this data.

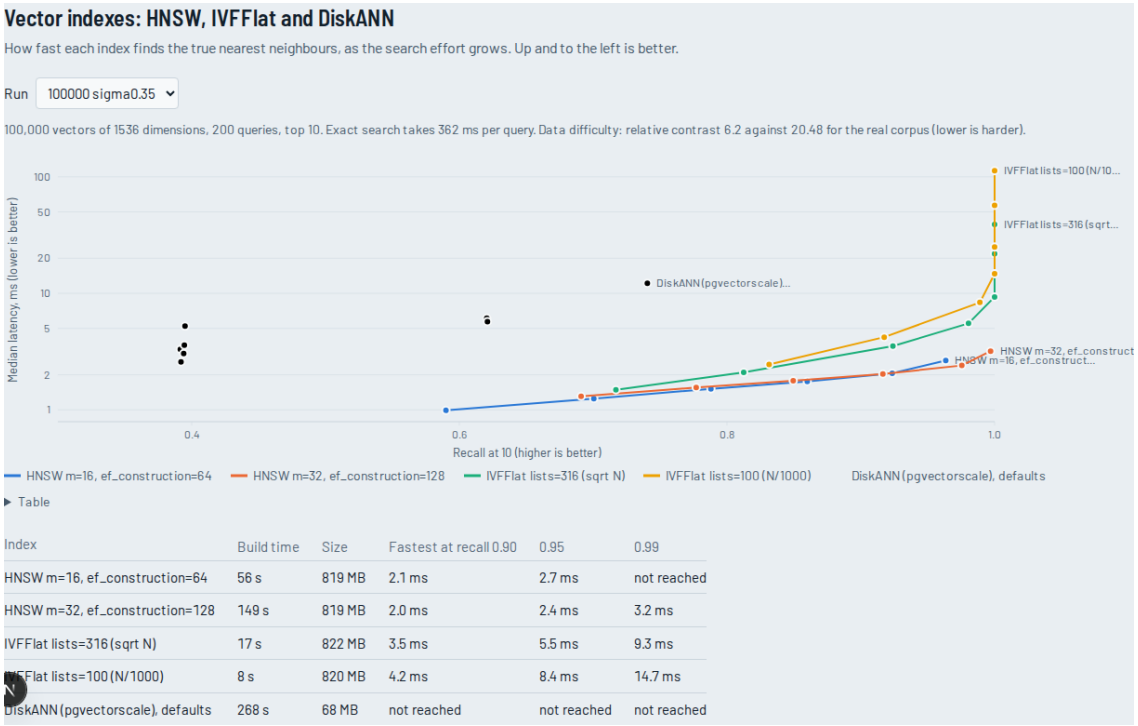


Figure 6: The same experiment on the Benchmarks page.

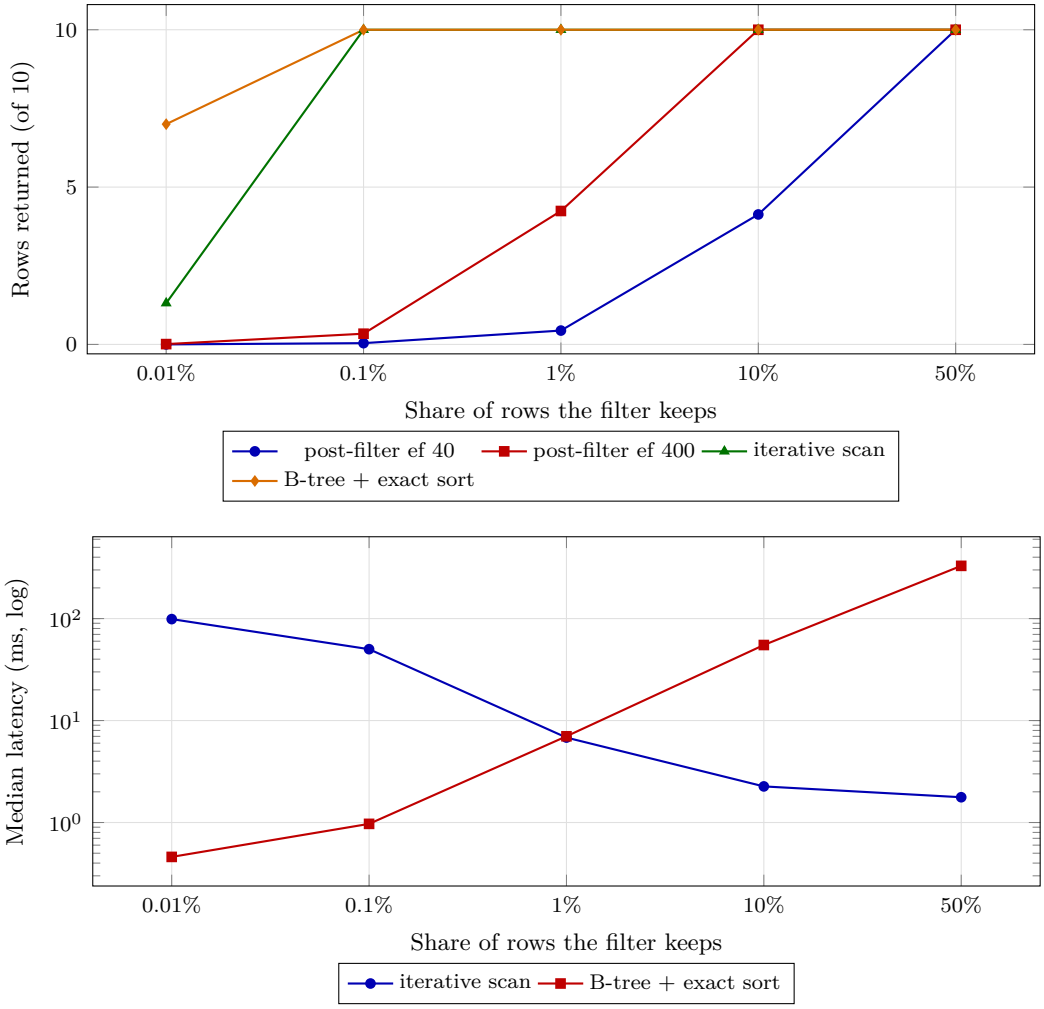
8.4 Does data difficulty change the winner?

The benchmark vectors are perturbed copies of real event embeddings, so the test asks whether synthetic easiness could favour an index. Holding $N = 50{,}000$ fixed and changing only the noise:

Index	RC10	p50 at recall ≥ 0.95	≥ 0.99
HNSW m=32, σ 0.35	6.0	1.47 ms	1.85 ms
HNSW m=32, σ 0.80	2.0	1.50 ms	2.57 ms
IVFFlat 224, σ 0.35	6.0	3.93 ms	3.93 ms
IVFFlat 224, σ 0.80	2.0	2.47 ms	3.76 ms
DiskANN, either σ	6.0 / 2.0	not reached	not reached

The real corpus has RC10 20.5, so both synthetic sets are harder than the real data, and the ranking is stable (HNSW m=32 fastest at 0.99 in both; DiskANN peaks at 0.845 and 0.889 recall). An earlier 50k-versus-100k comparison confounded size and noise; this controlled run replaced it.

8.5 Filtered search



Reading. Post-filtering with the default setting returns 0.0 rows of 10 when the filter keeps 0.1%, and 4.1 at 10%. Iterative scan restores the row count (not base recall) and costs 50 ms at 0.1%. Exact sort over a B-tree filter takes 1.0 ms at 0.1% but 55 ms at 10% and 329 ms at 50%. The crossover is near 1,000 rows.

Recall, filter correlated with the query	common	medium	rare
post-filter ef 40	0.92	0.66	1.00
post-filter ef 400	0.99	0.95	1.00
exact sort	1.00	1.00	1.00
partial HNSW per kind	0.91	0.66	1.00
list partitions + HNSW	0.93	0.66	1.00
DiskANN with labels	0.77	0.46	1.00

Table 5: A filter on the query's own event kind is harmless: every variant returned 10 rows, and partial indexes and partitions added nothing at this size. Builds: HNSW 62 s, partial indexes 62 s in total, partitions 101 s, DiskANN with labels 621 s.

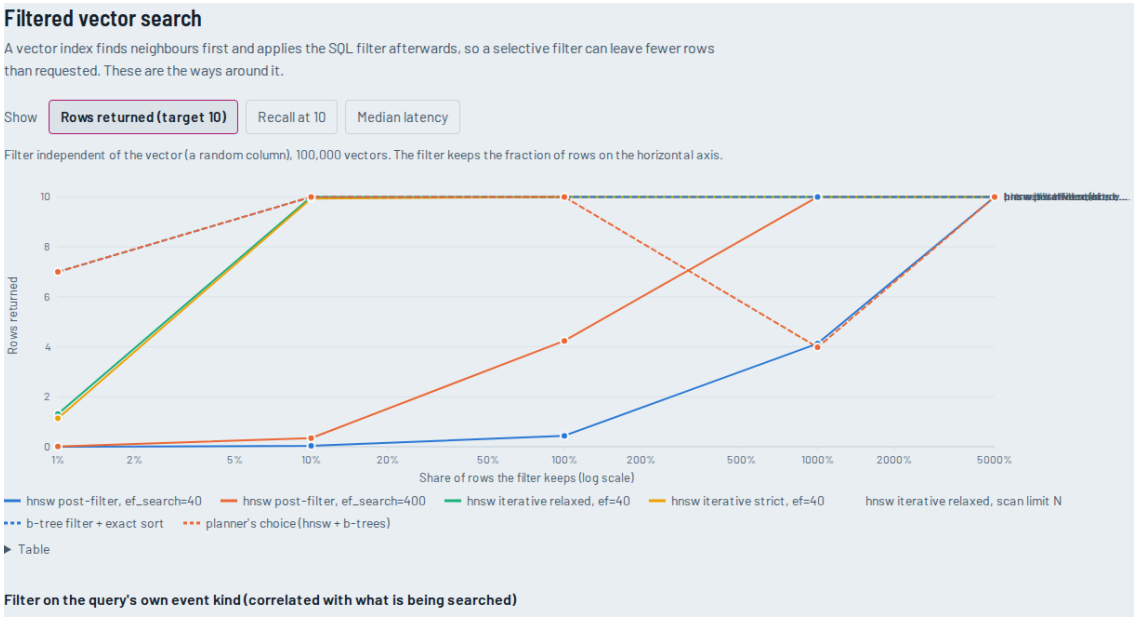


Figure 7: Filtered search on the Benchmarks page.

8.6 The planner picks wrong, so the search routes itself

Insight

With both indexes present and a filter keeping 10% of rows, the planner chose HNSW post-filtering: **4.0 of 10 rows**, recall 0.37, in 1.6 ms. It is fast and wrong, because a cost model estimates time, not completeness.

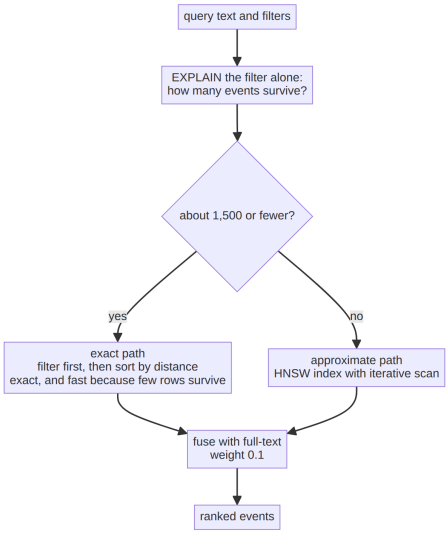


Figure 8: Router: estimate the filter's rows with `EXPLAIN`; exact path if about 1,500 rows or fewer, HNSW with iterative scan otherwise.

```
    est = estimated_rows(conn, filt, params)
    exact = est is not None and est <= EXACT_ROWS
    if info is not None:
```

```
        info["strategy"] = "exact (filter first, then sort by distance)" if exact else "approximate (HNSW index)"
        info["filter_estimated_rows"] = est
    conn.execute("SET hnsw.iterative_scan = relaxed_order")
    conn.execute("SET hnsw.ef_search = 100")
    filt_sql = "".join(f" AND {x}" for x in filt)
```

```
def estimated_rows(conn, filt: list[str], params: dict) -> int | None:
    """The planner's own estimate of how many events survive the filter (None when there is no filter)."""
    if not filt:
        return None
    sql = "EXPLAIN (FORMAT JSON) SELECT 1 FROM event e WHERE e.embedding IS NOT NULL" + "".join(f" AND {x}" for x
      in filt)
    plan = conn.execute(sql, {k: v for k, v in params.items() if k in ("kinds", "sev", "hw")}).fetchone()[0][0]["
     Plan"]
    return int(plan["Plan Rows"])
```

```
-- Ten nearest events by meaning
SELECT id, description FROM event
WHERE embedding IS NOT NULL
ORDER BY embedding <=> (SELECT embedding FROM event WHERE id = 7772)
LIMIT 10

Limit  (cost=168.04..173.82 rows=10 width=59) (actual time=40.817..40.934 rows=10 loops=1)
  Buffers: shared hit=159 read=125
  InitPlan 1
    ->  Index Scan using event_pkey on event event_1  (cost=0.28..2.50 rows=1 width=18) (actual time=0.028..0...
          Index Cond: (id = 7772)
          Buffers: shared hit=6
  ->  Index Scan using event_embedding_hnsw on event  (cost=165.54..2131.62 rows=3401 width=59) (actual time=...
        Order By: (embedding <=> (InitPlan 1).col1)
        Buffers: shared hit=159 read=125
Planning:
  Buffers: shared hit=16 read=1
Planning Time: 4.286 ms
Execution Time: 40.992 ms
```

9 Contribution 6: Guarded Natural-Language SQL

Plain English: a model that writes SQL could, if tricked, read server files or run a query that takes the server down. "It only runs SELECT" is not a safety boundary. The design uses four independent layers so defeating one is not enough.

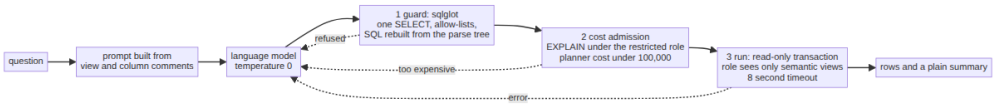


Figure 9: The guard pipeline.

1. **Guard** (`sqlglot`): exactly one SELECT, allow-listed tables and functions, no nested or recursive WITH; the SQL that runs is *regenerated from the parse tree*, so what was checked is what runs.
2. **Role**: a database role that can read only the documented views.
3. **Read-only transaction** with an 8 s statement timeout.
4. **Cost admission**: `EXPLAIN` first; refuse when the planner's estimate exceeds 100,000.

```
    root = statements[0]
    if not isinstance(root, (exp.Select, exp.SetOperation)):
        raise GuardError(f"Only SELECT queries are allowed, got {type(root).__name__.upper()}.")

    bad = next((n for n in root.walk() if isinstance(n, FORBIDDEN_NODES)), None)
    if bad is not None:
        raise GuardError(f"{type(bad).__name__.upper()} is not allowed in a query.")

    withs = [n for n in root.walk() if isinstance(n, exp.With)]
    if any(w is not root.args.get("with_") and w is not root.args.get("with") for w in withs):
        raise GuardError("WITH is only allowed at the top level of the query.")
    if any(w.args.get("recursive") for w in withs):
        raise GuardError("Recursive WITH is not allowed.")
    cte_names = {c.alias_or_name.lower() for c in root.find_all(exp.CTE)}
```

```
    used_tables: list[str] = []
    for t in root.find_all(exp.Table):
        if not isinstance(t.this, exp.Identifier):
            raise GuardError("Table-valued functions are not allowed in FROM.")
        name, schema = t.name.lower(), (t.db or "").lower()
        if t.args.get("catalog"):
            raise GuardError("Catalog-qualified table names are not allowed.")
        if not schema and name in cte_names:
            continue
        if schema and schema != policy.schema:
            raise GuardError(f"Schema '{schema}' is not available. Use only the {policy.schema} views: {', '.join
     (sorted(policy.tables))}.")
        if name not in policy.tables:
            raise GuardError(f"'{name}' is not available. Use only: {', '.join(sorted(policy.tables))}.")
        t.set("db", exp.to_identifier(policy.schema))  # qualify, so search_path never matters
        used_tables.append(name)
    if len(used_tables) > MAX_TABLES:
        raise GuardError(f"The query references {len(used_tables)} tables; the limit is {MAX_TABLES}.")
```

```
def run(pool, sql: str, mode: str = "semantic", max_cost: float = MAX_COST) -> Result:
    role = ROLES[mode]
    source, set_role = _connection_source(pool, mode)
    with source.connection() as c:
        try:
            with c.transaction():
                c.execute("SET TRANSACTION READ ONLY")
                if set_role:
                    c.execute(f"SET LOCAL ROLE {role}")
                c.execute(f"SET LOCAL statement_timeout = {TIMEOUT_MS}")
                c.execute("SET LOCAL idle_in_transaction_session_timeout = 10000")
                cost = float(c.execute("EXPLAIN (FORMAT JSON) " + sql).fetchone()[0][0]["Plan"]["Total Cost"])
                if cost > max_cost:
                    raise ExecError(f"The planner estimates this query is too expensive (cost {cost:,.0f}, limit
     {max_cost:,.0f}). "
                                    "Filter telemetry by flight_id and signal, or use signal_summary for
     comparisons across flights.")
                t0 = time.time()
                cur = c.execute(sql)
                fetched = cur.fetchmany(MAX_ROWS + 1)
                cols = [d.name for d in cur.description]
                run_ms = (time.time() - t0) * 1000
```

```
uavlog=# -- what the restricted role can and cannot do
ERROR:  permission denied for table flight
BEGIN
SET
 flights_visible
-----------------
             300
(1 row)

BEGIN
SET
ERROR:  permission denied for function pg_read_file
```

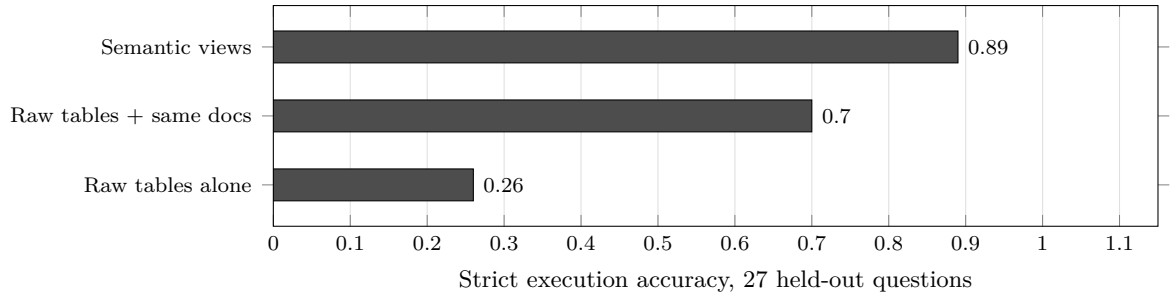
9.1 Hostile queries

```
MUST_REFUSE = {
    "drop": "DROP TABLE flight",
```

```
    "delete": "DELETE FROM semantic.flights",
    "update": "UPDATE semantic.flights SET board = 'x'",
    "insert": "INSERT INTO semantic.flights (log_id) VALUES ('x')",
    "stacked statements": "SELECT * FROM semantic.flights; DROP TABLE flight",
    "read file": "SELECT pg_read_file('/etc/passwd')",
    "read file, schema-qualified": "SELECT pg_catalog.pg_read_file('/etc/passwd')",
    "read file, upper case": "SELECT PG_READ_FILE('/etc/passwd')",
    "read file, comment between": "SELECT pg_read_file /* x */ ('/etc/passwd')",
    "copy program": "COPY flight TO PROGRAM 'id'",
    "dblink": "SELECT dblink('host=x', 'select 1')",
    "set_config": "SELECT set_config('role', 'uavlog', false)",
    "current_setting": "SELECT current_setting('is_superuser')",
```

46 hostile queries (stacked statements, `pg_read_file`, `COPY ... PROGRAM`, `dblink`, catalog access, sleeps, system settings) were each refused *for the intended reason*, and 18 legitimate queries still pass. Realistic questions plan under 5,000, while a telemetry self-join plans at 515 million and is refused before running. A fake model that obeys prompt injection is still stopped by the layers.

9.2 Evaluation: does the semantic layer help?



- **Documentation is the big effect**: 0.26 to 0.70 (12 questions gained, none lost, $p = 0.0005$).
- The views add 0.19 more, but the gain is **not significant** (6 gained, 1 lost, $p = 0.125$).
- Without documentation the model rarely fails loudly: 17 of 27 answers were confident and wrong. It never fabricated an answer to an unanswerable question.

Found by testing

The first run overstated the gap (0.65 against 0.24): my comparison treated numbers as strings, and my guard rejected a safe operator, which unfairly penalised the baseline. Both were fixed, the prompts frozen, and the evaluation re-run; both runs are kept in the result files.

```
  "id": "t11",
  "category": "events",
  "question": "What is the second most common event kind?",
  "answerable": true,
  "semantic_sql": "SELECT kind FROM semantic.events GROUP BY kind ORDER BY count(*) DESC OFFSET 1 LIMIT 1",
  "raw_sql": "SELECT c.key FROM event e JOIN concept c ON c.id = e.kind_id GROUP BY c.key ORDER BY count(*) DESC
     OFFSET 1 LIMIT 1"
},
{
```

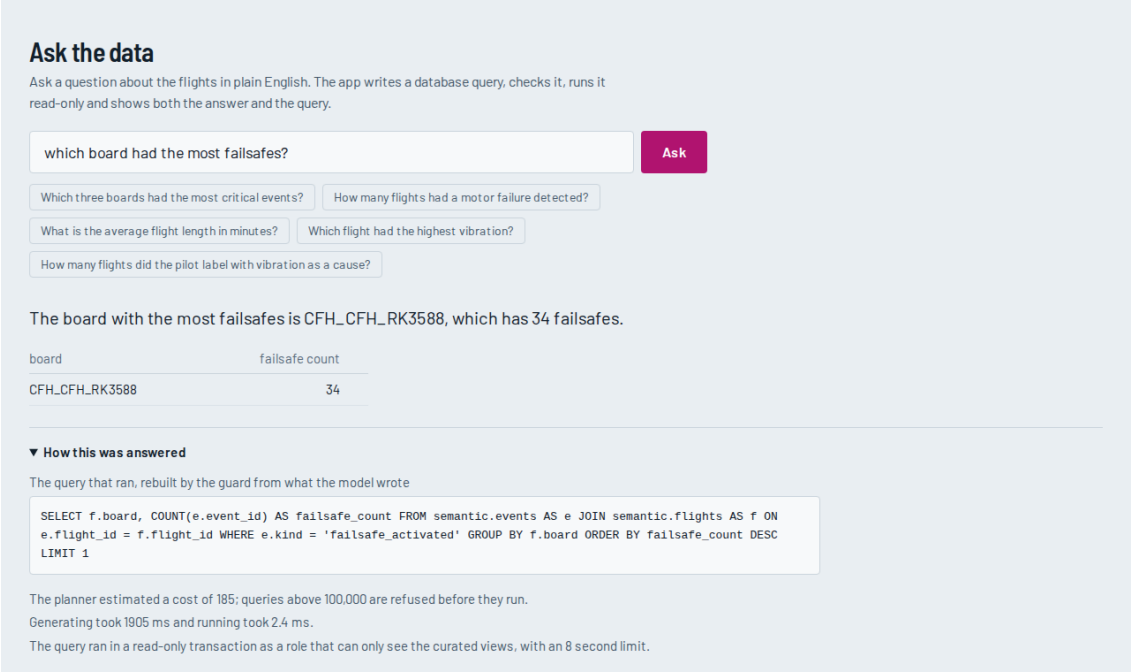


Figure 10: The Ask page: answer, generated SQL, planner cost and guard actions.



Figure 11: Search quality, NL-to-SQL and signature results on the Benchmarks page.

10 Contribution 7: Similar Flights by Behaviour

Plain English: find flights that behaved alike (signal statistics, event rates, time in each mode), not flights whose text looks alike. 145 features over 30 signals are scaled and reduced by PCA to a 32-dimension vector stored in `flight_signature`.

```
med = np.nanmedian(X, axis=0)
q75, q25 = np.nanpercentile(X, 75, axis=0), np.nanpercentile(X, 25, axis=0)
```

```
scale = np.where((q75 - q25) > 1e-9, q75 - q25, np.where(np.nanstd(X, axis=0) > 1e-9, np.nanstd(X, axis=0), 1.0))
Z = np.clip((X - med) / scale, -5, 5)
Z = np.where(np.isnan(Z), 0.0, Z)
pca_mean = Z.mean(axis=0)
_, _, vt = np.linalg.svd(Z - pca_mean, full_matrices=False)
k = min(DIMS, vt.shape[0])
comp = np.zeros((DIMS, len(names)))
comp[:k] = vt[:k]
S = (Z - pca_mean) @ comp.T
S /= np.maximum(np.linalg.norm(S, axis=1, keepdims=True), 1e-9)
```

The sanity check is the share of a flight's 5 nearest neighbours that run on the same board: chance 6.9%, scaled features 25.3%, the 32-dimension signature 25.0%. Compression to 32 dimensions (86% of the variance) loses nothing measurable. This is a sanity check rather than ground truth, since hardware influences sensor scale.

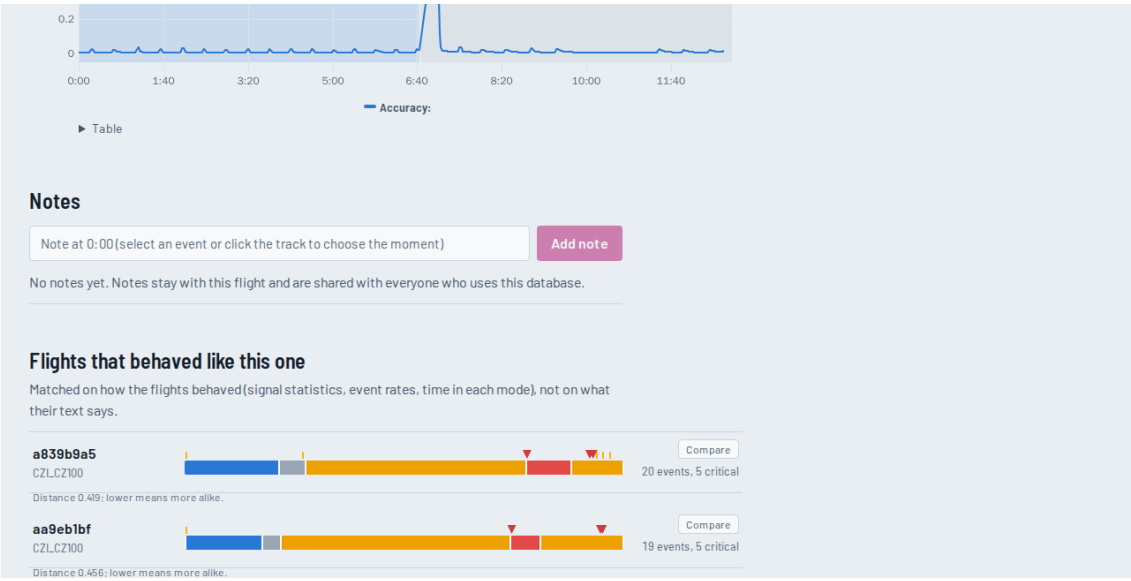


Figure 12: Notes and "flights that behaved like this one" on a flight page.

11 The Application

The interface is a Next.js application with a vertical navigation bar (Explore, Analyze, Platform), a slim top bar with a command palette (Ctrl+K), and light as the default theme. Every page below reads live from the API; none of the numbers are typed in.

11.1 Exploring flights

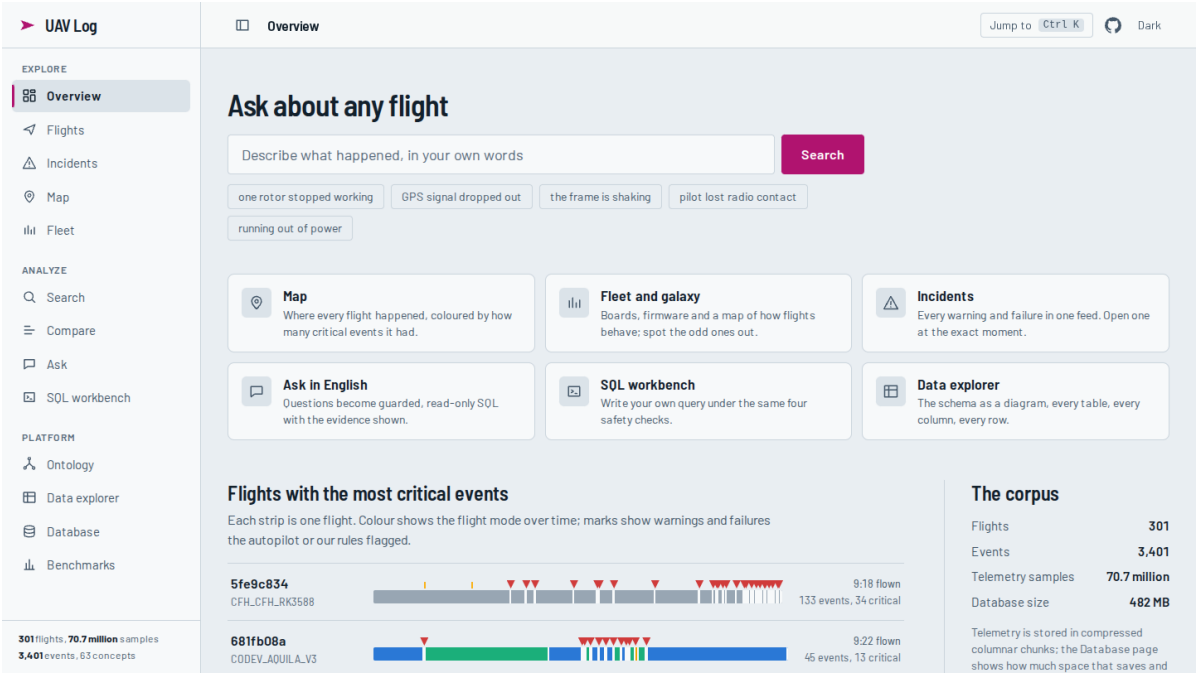


Figure 13: Overview: search by meaning, entry points to every feature, and flights ranked by critical events.

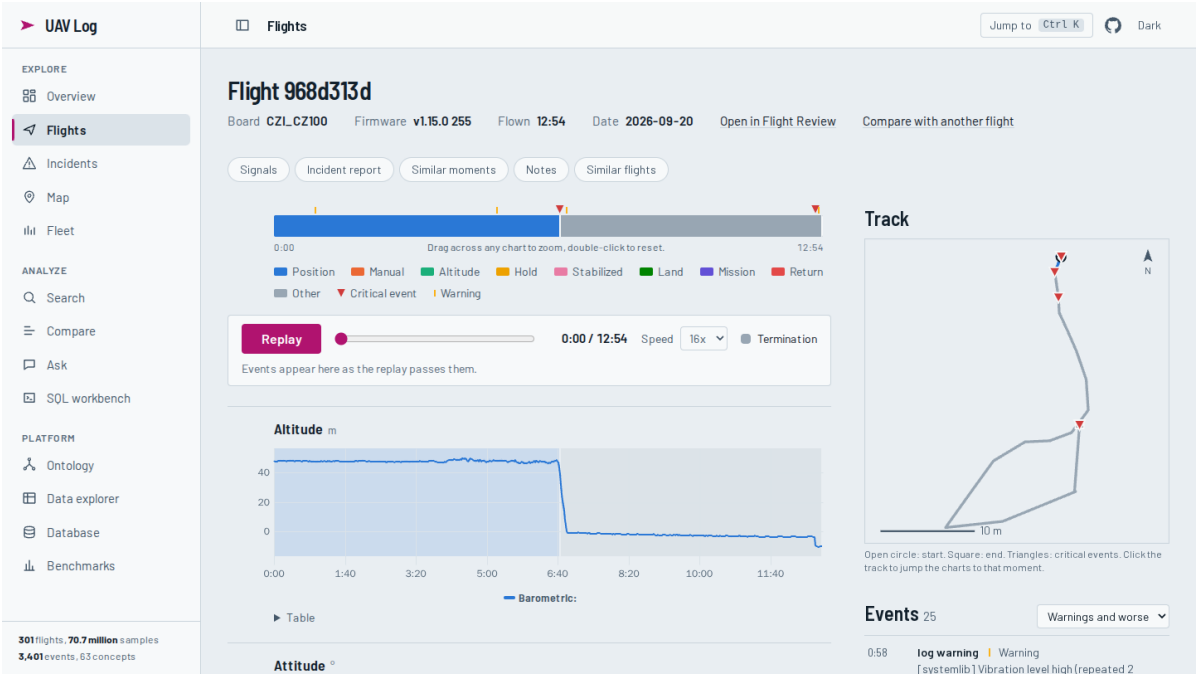


Figure 14: One flight: mode ribbon with event marks, replay bar, synchronized charts, ground track and event list.

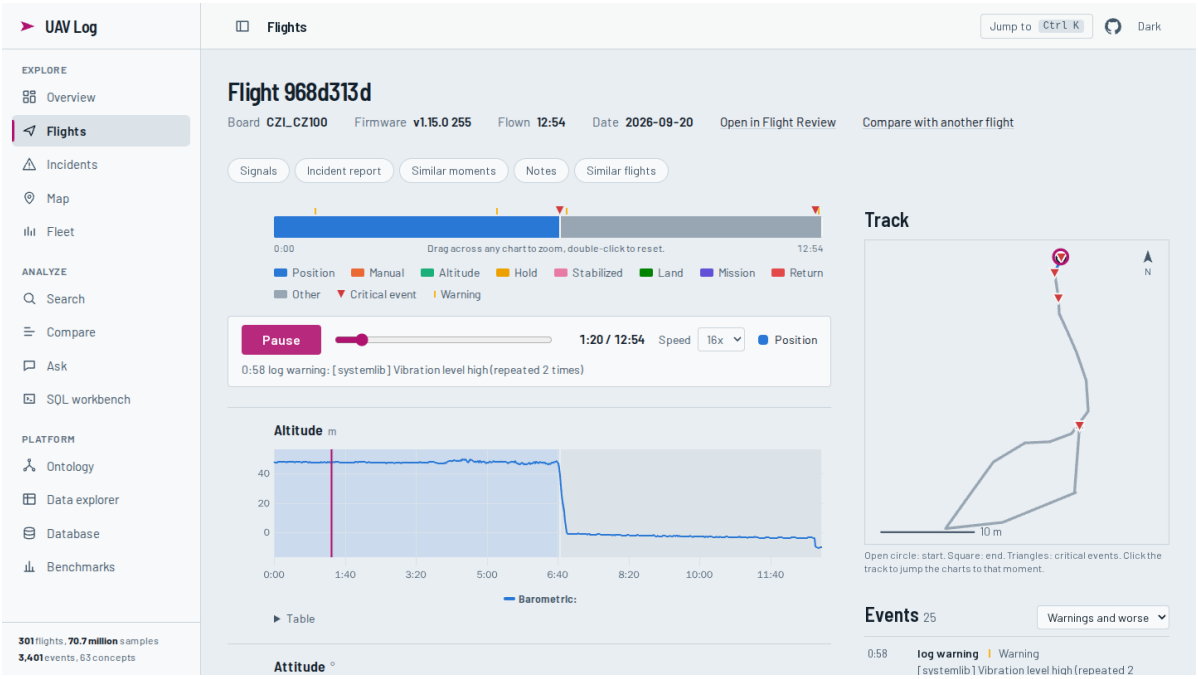


Figure 15: Flight replay: the time cursor sweeps every chart and the track at 1x to 64x; events surface as it passes them.

11.2 Incident reports with citations

Plain English: the database gathers numbered facts about the flight; a language model turns them into a report; every sentence must cite the facts it used, and a sentence with no valid citation is deleted. The evidence (events, signals unusual against the whole corpus, similar flights) is computed by SQL (`gather_evidence`); the report is cached in the `investigation` table.

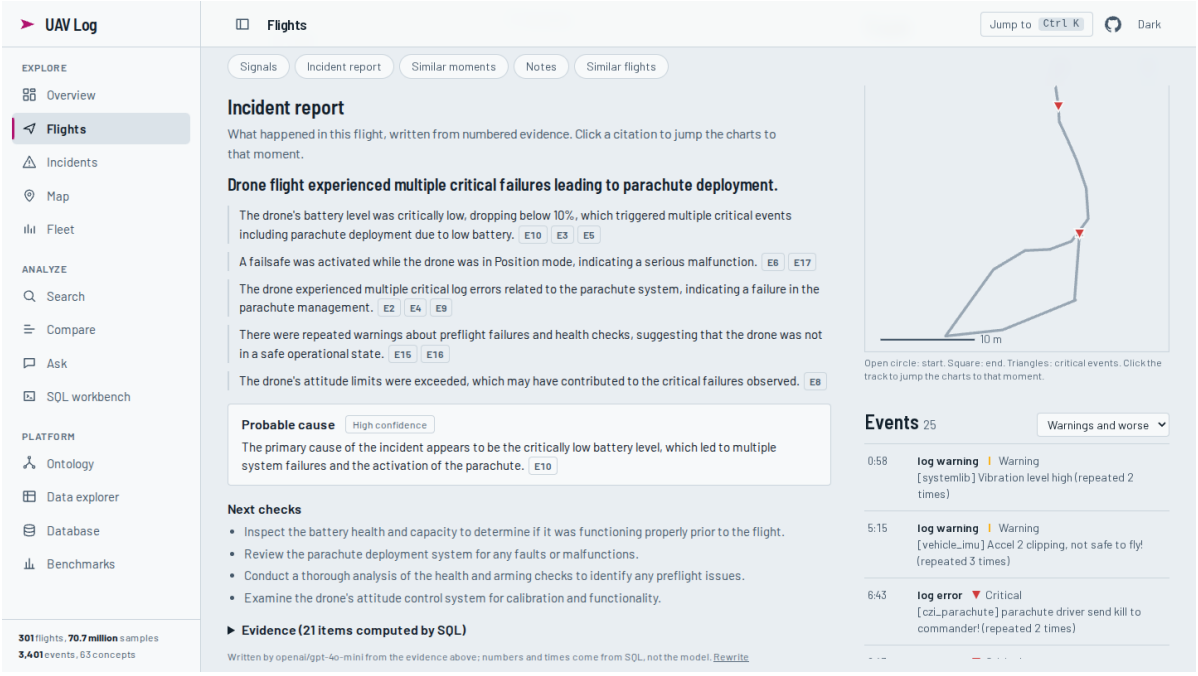


Figure 16: Incident report: findings with citation chips (E3, E6, ...) that jump the charts to the cited moment; the full evidence list is expandable.

11.3 Similar moments

Each flight is cut into overlapping 10-second windows (hop 5 s). A window is described by the mean, standard deviation, minimum, maximum and slope of 30 canonical signals, robust-scaled, reduced by PCA to 32 dimensions (84% of the variance) and stored in `signal_window` with an HNSW index (18,238 windows). Searching a moment returns the best window in each other flight.

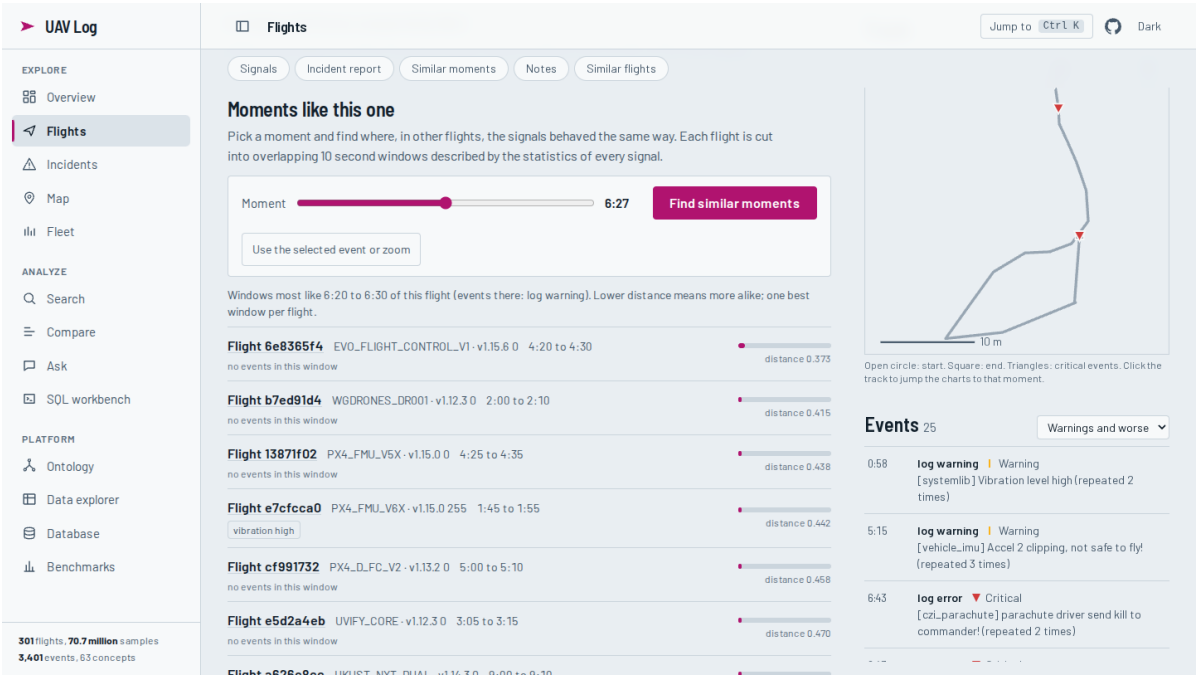


Figure 17: Similar moments: pick a moment, find where other flights behaved the same way.

Event kind	Queries	Random base rate	Precision at 10	Lift
landing	60	0.012	0.500	43.4×
disarm	60	0.007	0.233	33.1×
takeoff	60	0.022	0.705	32.6×
arm	35	0.005	0.123	22.5×
vibration high	15	0.022	0.247	11.2×
ekf innovation reject	32	0.016	0.166	10.5×
attitude limit exceeded	26	0.009	0.069	7.5×
mode change	60	0.064	0.170	2.7×
failsafe activated	60	0.013	0.023	1.7×
log error	60	0.089	0.147	1.6×
battery low	41	0.051	0.078	1.5×
log warning	60	0.347	0.377	1.1×

Table 6: A result counts when it overlaps an event of the same kind as the query window. HNSW with iterative scan returns the same results as exact search (recall 0.99, p50 0.3 ms). Without iterative scan 7% of queries returned fewer than 10 rows, because the nearest windows are the query flight's own neighbours and the filter removes them after the index scan: the filtered-search pitfall, on real data.

11.4 Fleet views

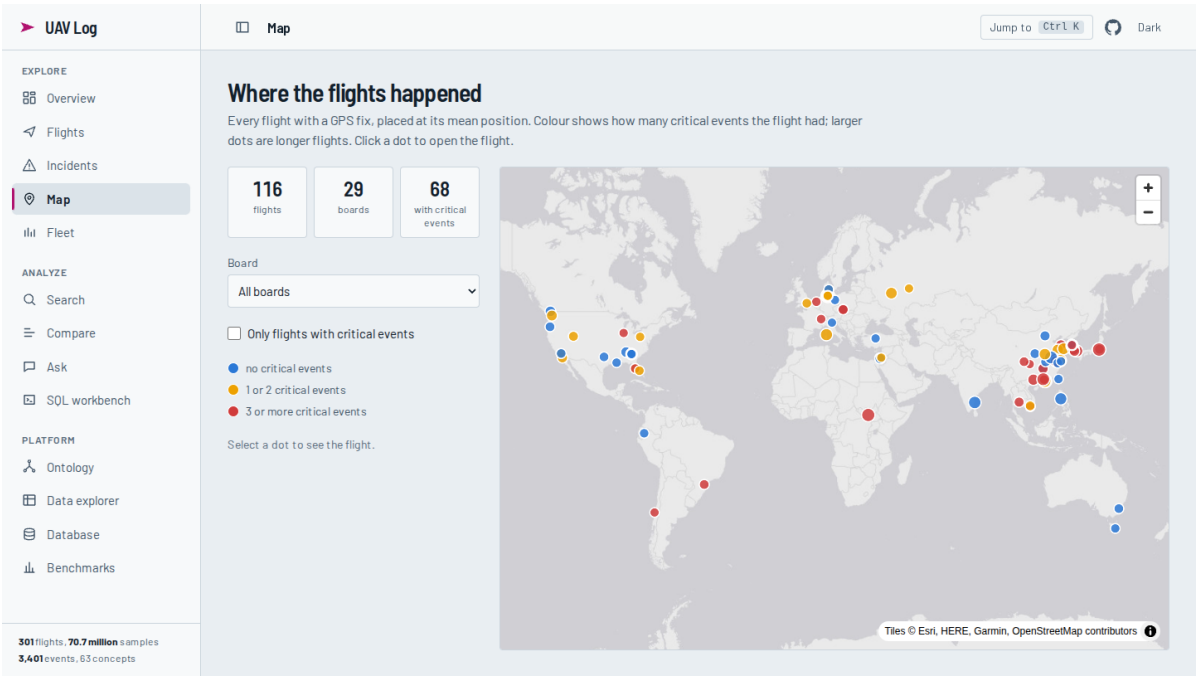


Figure 18: Map: every flight with a GPS fix at its mean position, coloured by critical events (from the signal summary view).

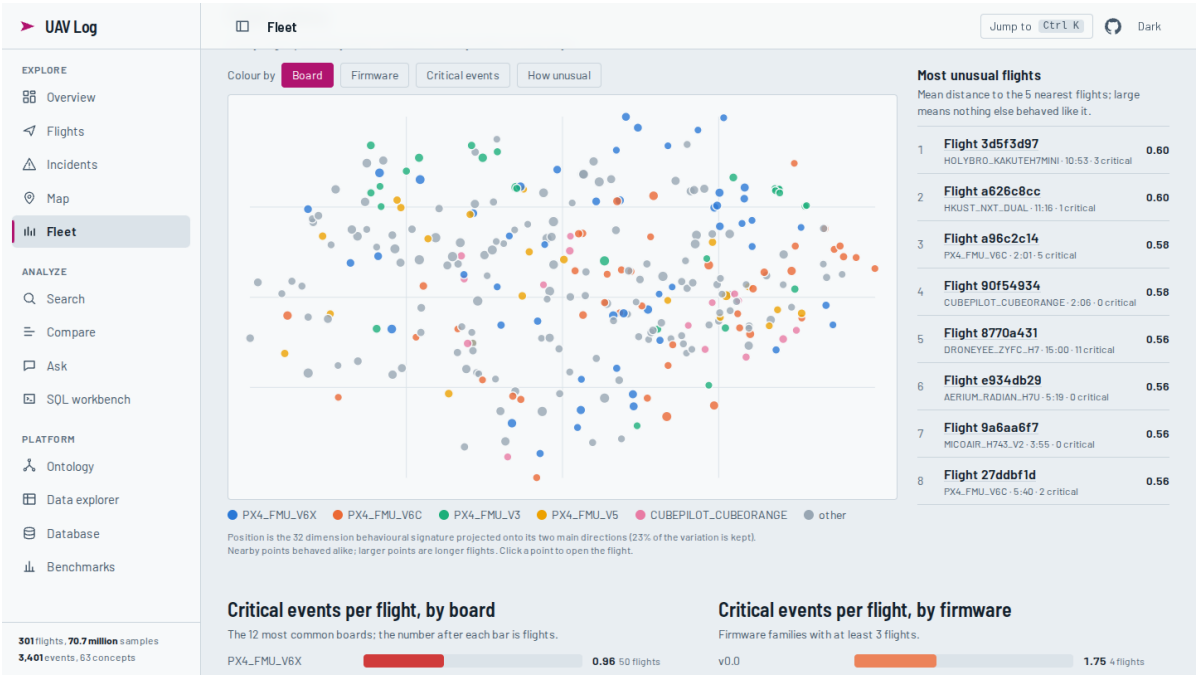


Figure 19: Flight galaxy: the 32-dimension behavioural signature projected onto its two main directions (23% of the variation); colour by board, firmware, critical events or unusualness (mean distance to the 5 nearest flights).

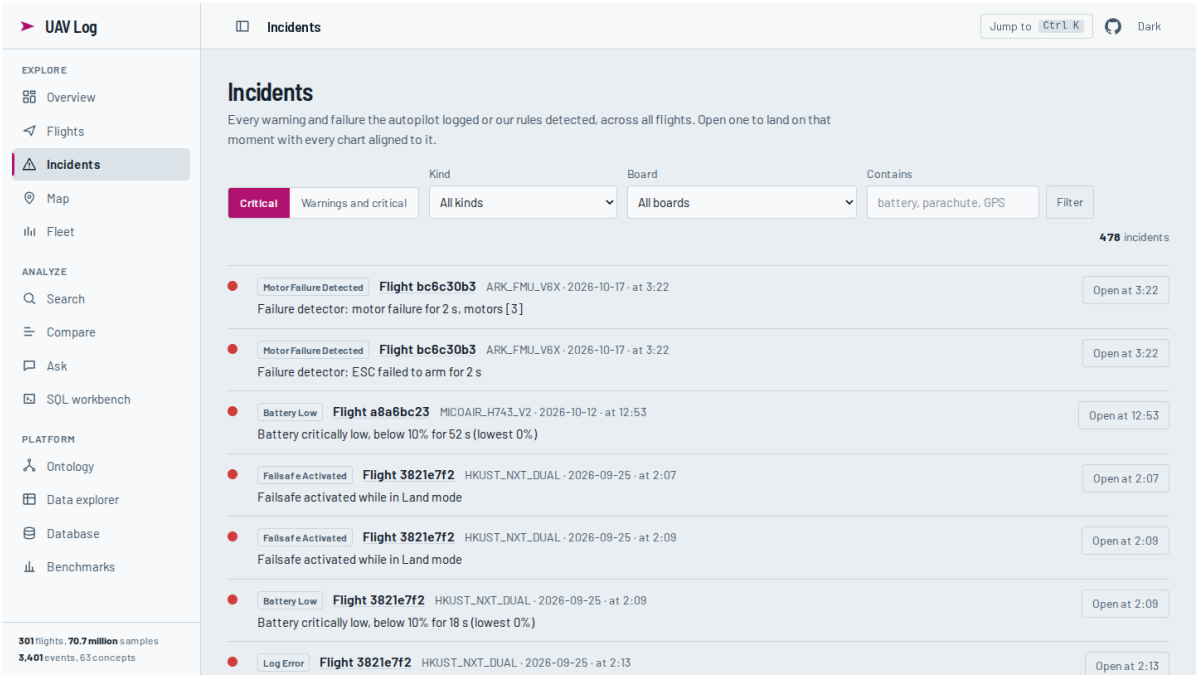


Figure 20: Incidents: every warning and failure across flights, filterable by severity, kind, board and wording, with a link to the exact moment.

11.5 Asking questions

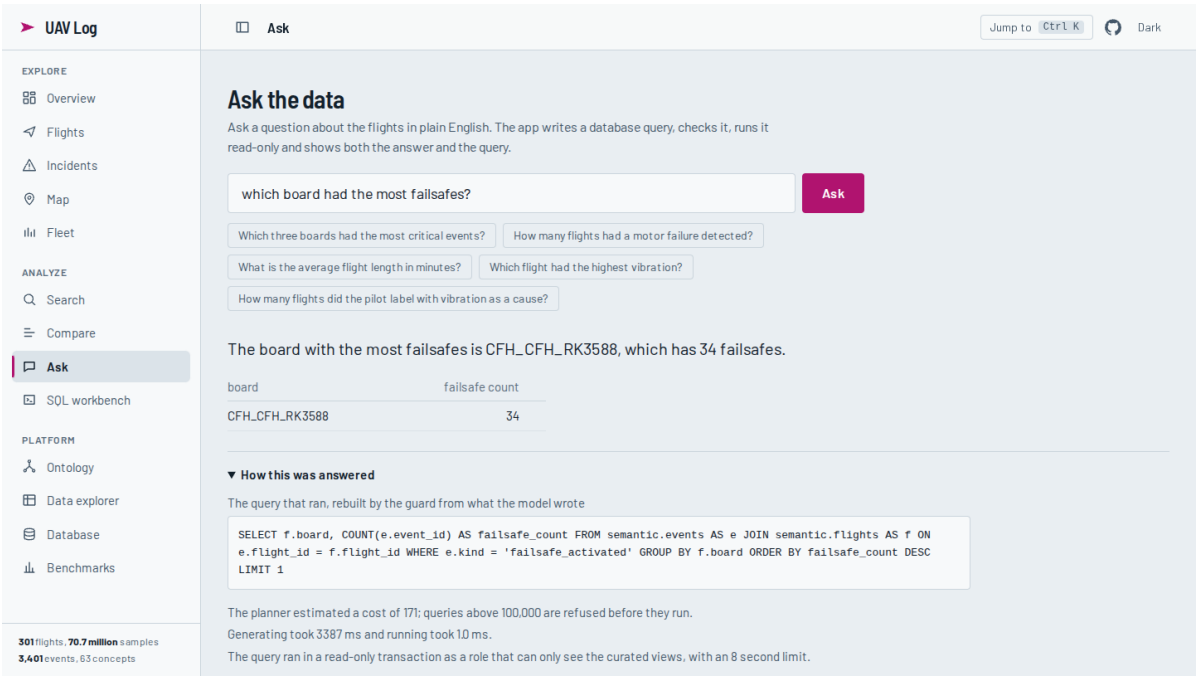


Figure 21: Ask: natural language to guarded SQL; the SQL, the planner's cost and each guard action are shown.

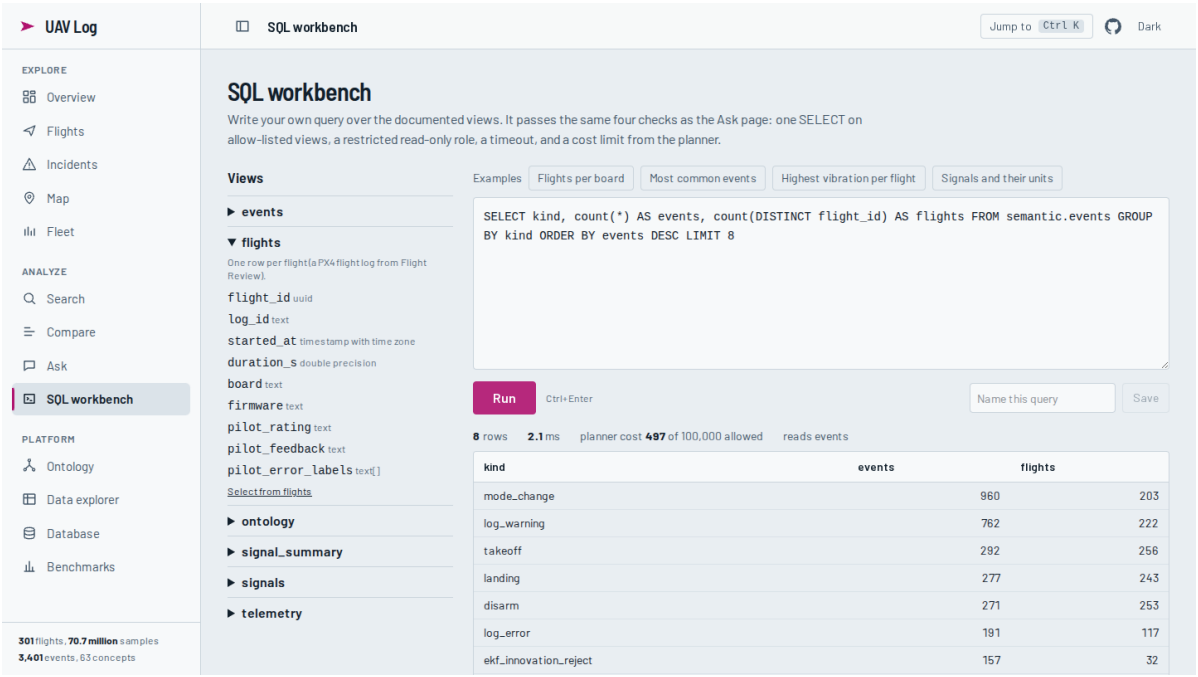


Figure 22: SQL workbench: hand-written SQL under the same four checks (guard, restricted role, read-only transaction with a timeout, planner cost admission); a schema browser and saved queries.

11.6 Looking inside the database

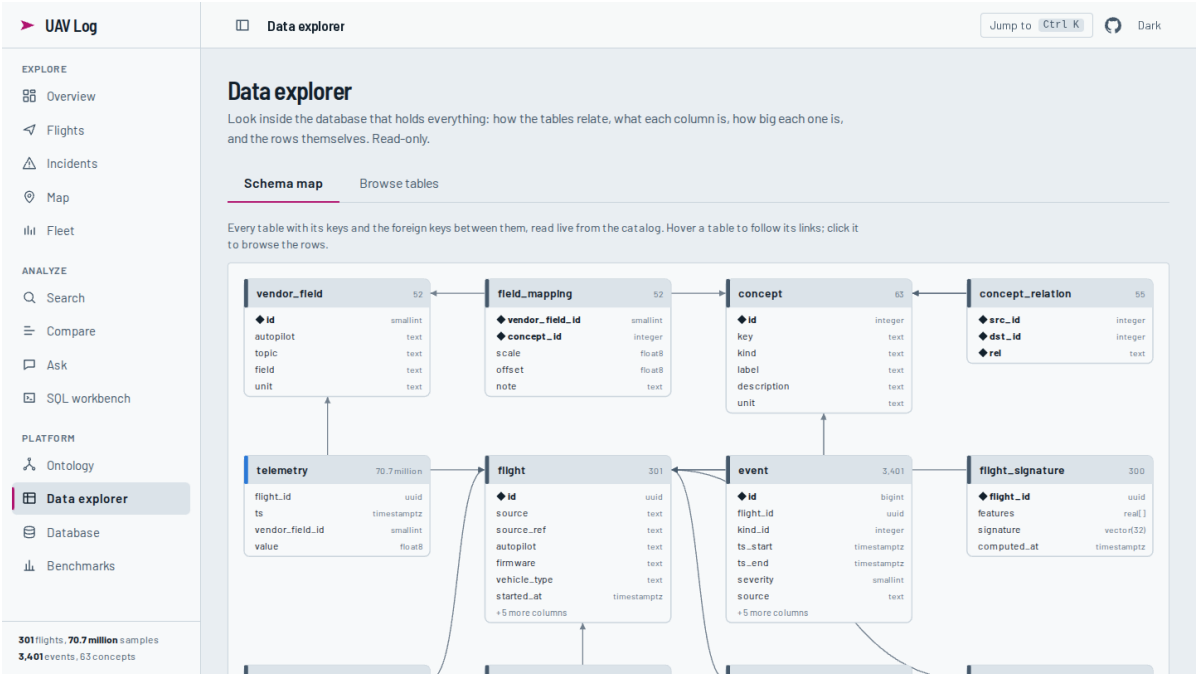


Figure 23: Data explorer, schema map: tables, keys and foreign keys drawn live from the catalog.

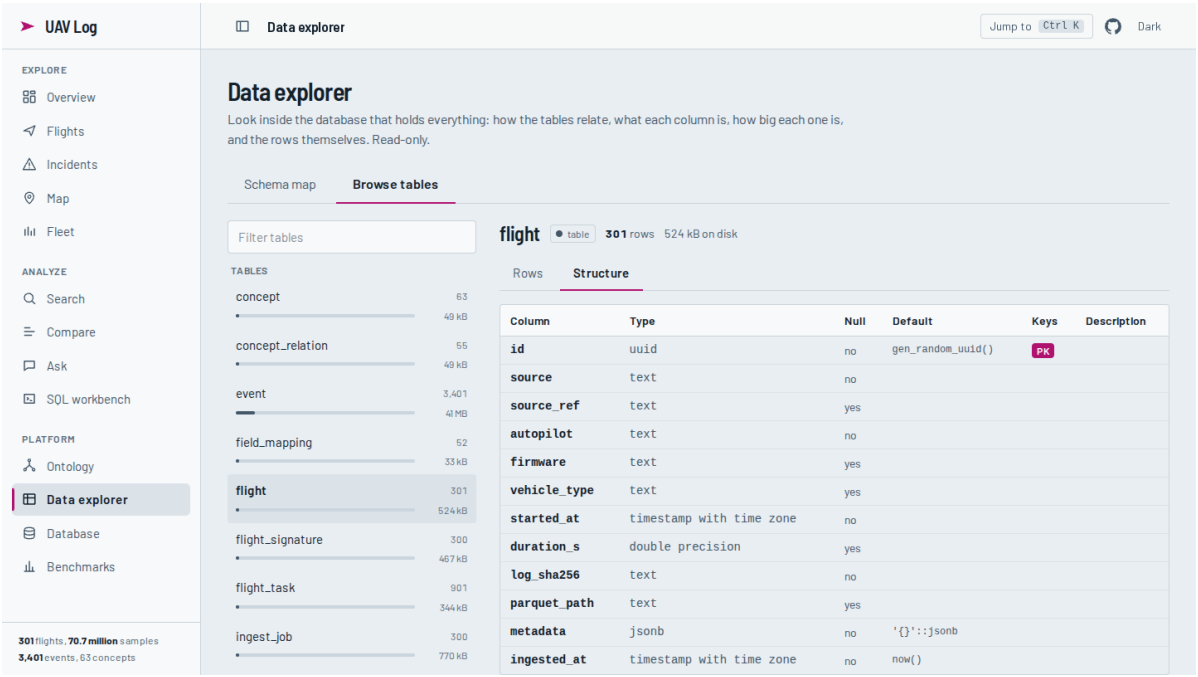


Figure 24: Data explorer, structure: columns, keys, indexes with sizes, and the hypertable's chunk and columnstore settings.

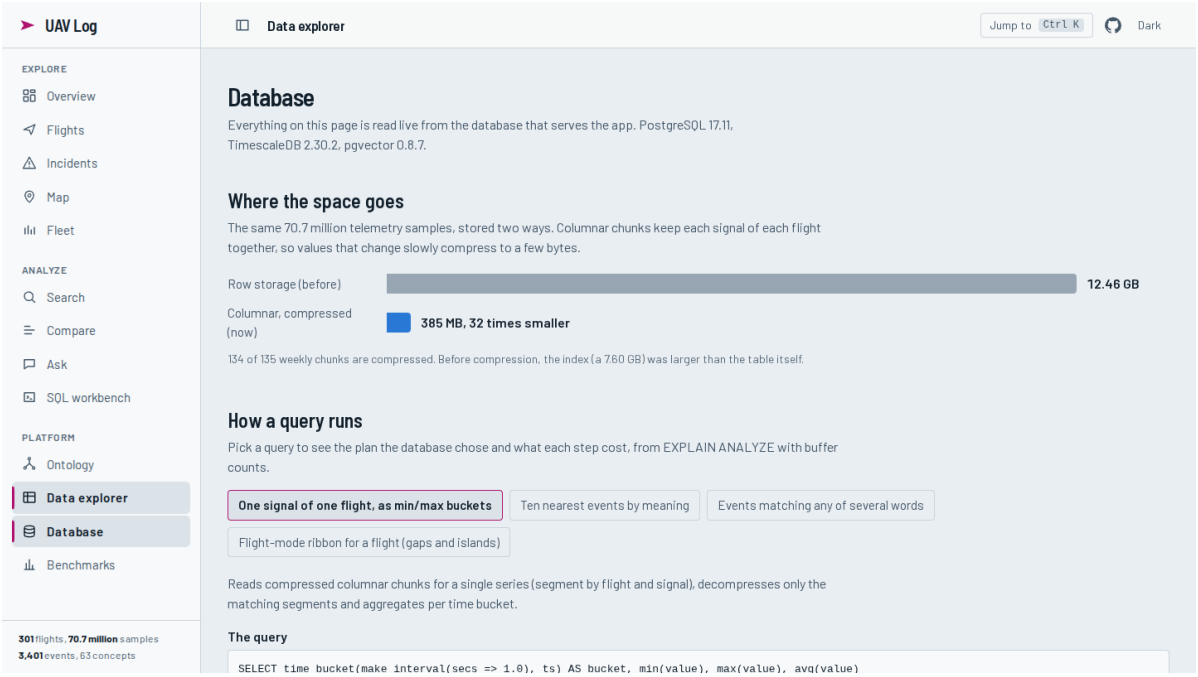


Figure 25: Database page: space used (row storage against columnar) and a selectable query with its `EXPLAIN ANALYZE`.

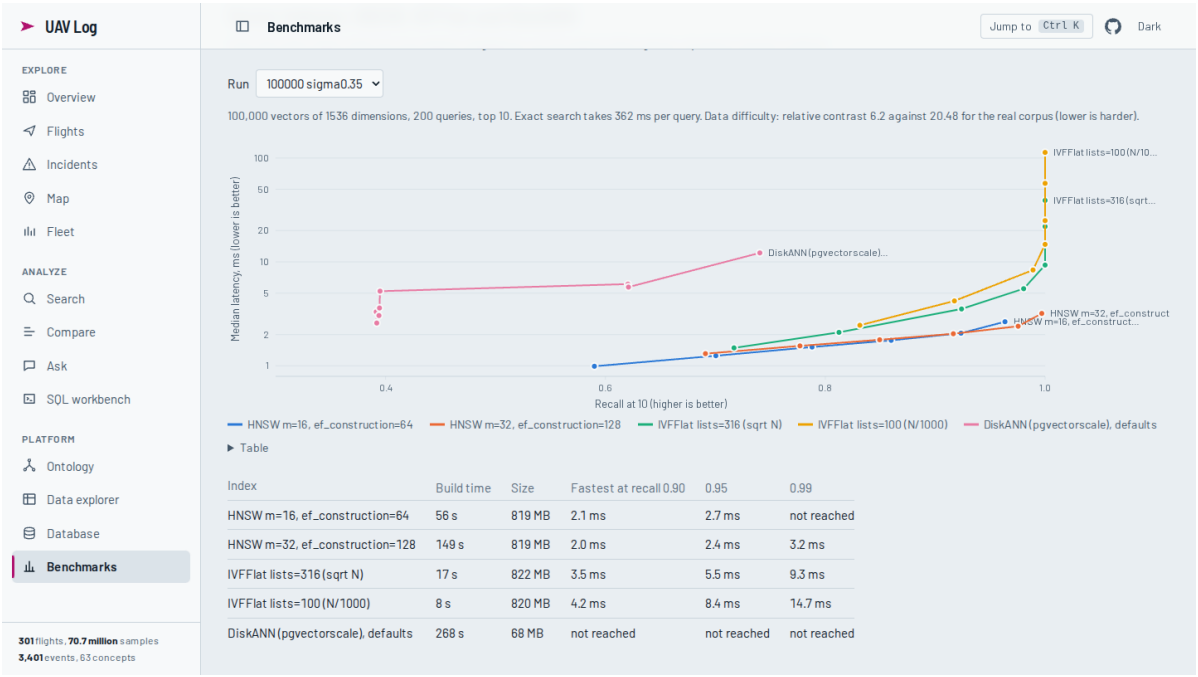


Figure 26: Benchmarks page: every experiment in this report is a section drawn from the saved result files.

12 Results Summary

Question	Measured answer
RQ1 storage	12,459 MB to 385 MB (32×), 176 to 5.4 bytes per row; segmentby (flight, signal) and chunk size set cross-flight speed (up to 95× faster).
RQ1 engines	DuckDB wins full scans and point slices on its native file; the columnstore wins pruned access and the summary build; Postgres keeps concurrency, roles and vectors.
RQ2 index	HNSW m=32: 0.95 recall in 2.4 ms (exact: 362 ms); IVFFlat builds up to 17× faster; DiskANN is 12× smaller but less accurate.
RQ2 filtering	Post-filtering returns 0.0 of 10 rows at 0.1%; exact sort wins when selective, HNSW when not; a router using the planner estimate chooses.
RQ2 fusion	Plain RRF hurts (0.600 against 0.710); full text adds nothing measurable.
RQ3 NL-to-SQL	0.89 with views, 0.70 with documentation alone, 0.26 without; four layers refuse 46 of 46 hostile queries.
RQ4 detection	90–100% of injected faults found once they last 5 s; battery-low raises 23% false alarms on controls.

13 Theoretical Comparison Table

Choice	Strength	Weakness	Used here for
Rowstore + B-tree	Fast point reads, simple	176 B/row, slow scans, large index	Baseline
Columnstore	32× smaller, pruned reads	Updates cost more (partial chunks)	Telemetry
DuckDB / Parquet	Fast scans, no server	Single writer, no roles	Comparator
HNSW	Fastest queries at high recall	Slow build, memory-heavy	Event search
IVFFlat	Fast build	Slower queries, needs tuning	Comparator
DiskANN	12× smaller	Lower recall here	Comparator
Post-filter	Simple	Under-returns when selective	Avoided
Iterative scan	Restores row count	50 ms at 0.1%	Unselective filters
Exact sort	Exact, cheap if selective	Linear in matches	Selective filters
Plain RRF	Needs no score calibration	Hurt quality here	Measured, not used

Table 7: Design alternatives and where each was used.

14 Design Decisions

The decision log (`docs/decisions.md`) records what, why and the alternatives for each choice. The ones that shaped the database:

D1 One Postgres with TimescaleDB and pgvector as the core store (polyglot alternatives rejected for operational weight and because they hide the planner trade-offs studied here).

D2 Narrow telemetry table with only ontology-mapped signals; everything else stays in Parquet.

D3 Columnstore with `segmentby (flight_id, vendor_field_id)`, `orderby ts`.

D4 Compression as an explicit, repeatable stage that also handles partial chunks.

D6 Data-quality guards at ingest (implausible device clocks).

D7 Events from log transitions and threshold rules.

D11–D13 Hybrid retrieval with RRF; measured result that vector wins and no absolute relevance cutoff exists.

D15 Robust flight duration after one corrupt timestamp.

D16 NL-to-SQL guarded, read-only and measured with and without the semantic layer.

D18 Filtered vector search routed by the planner's row estimate (exact sort at or below about 1,500 rows, HNSW with iterative scan above).

D19 HNSW m=32 for event embeddings; IVFFlat and DiskANN kept as comparators; controlled difficulty check.

D20 Columnstore physical design and the TimescaleDB against DuckDB comparison.

D21 Fault injection on the event detectors, with the battery-low rule as the weakest result.

D22 Flight signatures, and the ArduPilot log loaded through the same ontology.

D23 Signal windows for similar moments; cited incident reports (evidence by SQL, prose by a model); read-only data explorer and guarded workbench.

15 Glossary

Hypertable Time-partitioned table in TimescaleDB.

Chunk One time-range partition of a hypertable.

Columnstore Compressed column-wise storage of older chunks.

Continuous aggregate Incrementally maintained rollup (here 1-second min, max, mean).

Embedding A list of numbers representing meaning; here 1,536 dimensions for events and 32 for flight signatures.

HNSW / IVFFlat / DiskANN Approximate nearest-neighbour index families.

Recall@k Fraction of the true top-k returned.

nDCG@10 Ranking quality over the top 10.

RRF Reciprocal rank fusion of several rankings.

Ontology Concepts and relations; here stored in relational tables.

Gaps and islands SQL pattern that finds runs of consecutive rows.

SKIP LOCKED Row-locking mode that skips rows another transaction holds; used by the job tables.

Execution accuracy Share of generated queries whose result equals the gold result.

RC10 Relative contrast of the 10th neighbour; lower means harder data.

Parquet Open columnar file format used for the raw layer.

16 Frequently Asked Questions

Why one database instead of several specialised ones? Transactions, roles, joins to the ontology and vectors then live together, and the planner trade-offs become the object of study. DuckDB won some scans in the comparison but gives up concurrent writers and roles.

Why a narrow telemetry table? New signals and new autopilots then need no schema change, and the ontology can map any vendor field. The cost is more rows and a join to name signals; the columnstore turns the extra rows into 5.4 bytes each.

Is 300 flights enough? It is a stated limit. Storage results are measured on 70.7 million rows; vector benchmarks use 50,000 and 100,000 perturbed vectors made harder than the real data. A larger run on 10,000 or more logs is the next step and every volume is configurable.

Why did hybrid search lose? On 65 queries vector similarity already handles paraphrases, and plain fusion lets weak keyword hits displace good vector hits. With a small keyword weight it ties; no evidence was found that it helps.

Why did the planner choose the wrong plan? Its cost model estimates time, not result completeness. HNSW post-filtering looks cheap but returns 4 of 10 rows. The router avoids depending on that estimate for correctness.

Is the NL-to-SQL "safe"? It is defended in four independent layers and tested with 46 hostile queries. The remaining risk is correctness (confident wrong answers), which is why the SQL and its cost are shown with every answer.

The views added 0.19 accuracy but it is "not significant". Is it useful? The direction is positive (six questions gained, one lost) but 27 questions cannot separate it from chance ($p = 0.125$). The clear effect is documentation.

What is the weakest result? The battery-low detector: it is silent below its minimum duration and raises false alarms on 23% of unmodified flights.

17 Conclusions

- A single PostgreSQL instance holds the time series, the ontology, the vectors and a guarded natural-language interface.
- Physical design cut telemetry from 12.5 GB to 385 MB (32×); measured, not assumed.
- For retrieval, filtering strategy mattered more than index choice, and a self-routing query beat any fixed plan.
- Documentation, more than views, drives NL-to-SQL accuracy; four independent layers make it safe.
- Detectors find injected faults reliably from 5 s, with a known weakness in the battery rule.

Limits. 300 flights, one machine, one model, author-written questions, perturbed vectors for index benchmarks, heuristic event thresholds. **Next.** The 10,000-log run, ArduPilot at scale, a concurrency benchmark.